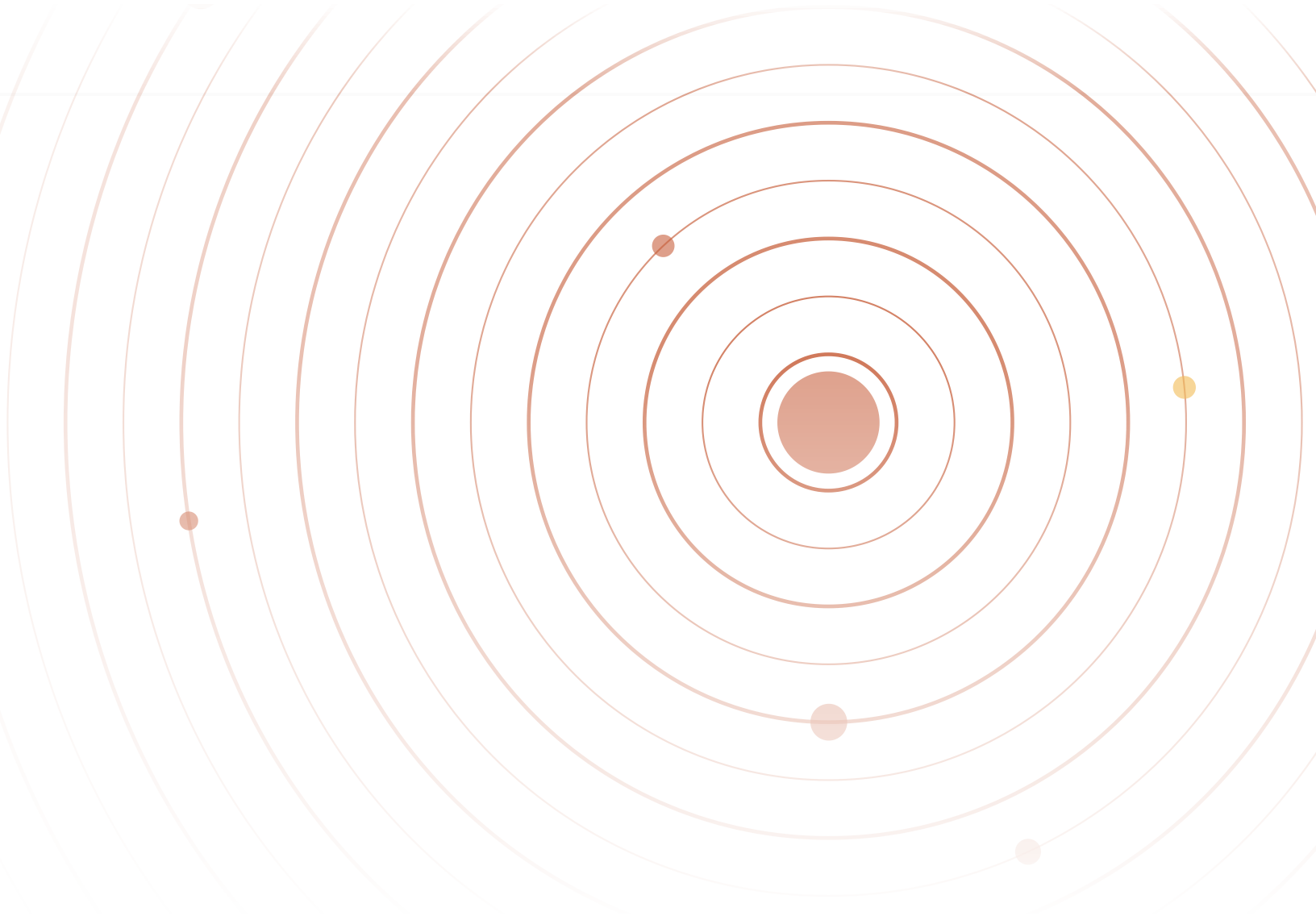


A BEGINNER'S MANUAL

Claude Code

Your first AI coding partner, from installation to everyday mastery

For complete beginners • October 2026 edition • Print edition · US Letter

GUIDES FOR EVERYTHING

Claude Code

Your first AI coding partner, from installation to everyday mastery

Claude Code is an AI assistant from Anthropic that can read your project, write and edit files, run commands, and explain everything it does in plain English. You talk to it the way you'd talk to a helpful colleague, and it does the work.

This manual assumes you have **never used a terminal before**. Each chapter builds on the one before it. By the end you'll have installed Claude Code, built and changed real files with it, learned to control what it is allowed to do, and set it up to remember how you like to work.

HOW TO READ THIS MANUAL

- **Tip** — a shortcut or good habit.
- **Note** — useful background.
- **Warning** — easy to get wrong.
- **Danger** — can lose data, money, or access.
- `$ command` — text you type, exactly as shown.
- Settings › Privacy — a menu path to click.

New words appear in **bold** the first time and are collected in the Glossary at the back.

Facts, prices, and menu paths were checked against official documentation in October 2026. Software changes quickly — when something on screen doesn't match, trust the product's current docs.

Contents

01 **Welcome to Claude Code** 6

1.1 What is an “agent”? 6

1.2 What can you do with it? 7

1.3 Where Claude Code runs 7

1.4 What you need 8

02 **Terminal Basics in Ten Minutes** 9

2.1 Opening a terminal 9

2.2 Folders and paths 10

2.3 The five commands you need 10

2.4 Copying and pasting in a terminal 11

03 **Install and Sign In** 12

3.1 Install with the native installer 12

3.2 Choose a plan 14

3.3 Log in for the first time 14

3.4 Health check and updates 15

04 **Your First Session** 16

4.1 The Claude Code screen 16

4.2 Build a small website 17

4.3 Ask questions about existing code 17

4.4 Approving, interrupting, and undoing 18

4.5 Ending and resuming 18

05 **Talking to Claude Effectively** 19

5.1 Five habits of great prompts 19

5.2 Plan mode: look before you leap 20

5.3 Share what you see 20

5.4 Ask a side question 20

5.5 Keep conversations focused 21

06 **Permissions and Safety** 22

6.1 Permission modes 22

6.2 Allow, ask, and deny rules 23
6.3 Where settings live 24
6.4 Safety nets 24

07 **Memory: Teaching Claude Your Project** 25
7.1 CLAUDE.md files 25
7.2 Create one with /init 26
7.3 What to put in it 26
7.4 Auto memory 27

08 **Everyday Commands and Shortcuts** 28
8.1 Slash commands 28
8.2 Keyboard shortcuts 29
8.3 Starting Claude Code from the terminal 30

09 **Extending Claude Code** 31
9.1 MCP: connect to other apps 31
9.2 Skills: your own slash commands 32
9.3 Subagents: specialist helpers 33
9.4 Hooks: automatic actions 33
9.5 Plugins: install bundles 34

10 **Beyond the Terminal** 35
10.1 The desktop app 35
10.2 VS Code and Cursor 35
10.3 Claude Code on the web and phone 35
10.4 Scripts and automation 36

11 **Costs and Limits** 37
11.1 How usage works 37
11.2 Make your usage go further 37

12 **Troubleshooting** 38

Quick Reference 39

13 **The Complete Slash-Command Reference** 40
13.1 Three kinds of slash command 41
13.2 /clear: start fresh 41
13.3 /compact: shrink without forgetting 42

13.4 /context: see what's using space 42
13.5 /rewind: the undo button 42
13.6 /resume, /branch, /fork, and /rename 43
13.7 /model and /effort 43
13.8 /init and /memory 43
13.9 /permissions, /sandbox, and /fewer-permission-prompts 44
13.10 Review and quality commands 44
13.11 Housekeeping commands 45
13.12 Recipes: commands that work well together 45
13.13 Every documented command, A–Z 45

14 Command-Line Flags and Shortcuts Reference 52

14.1 The ten flags beginners actually use 53
14.2 Terminal subcommands 53
14.3 Every startup flag 55
14.4 Every keyboard shortcut 58

15 Real-World Workflows, Step by Step 62

15.1 Workflow 1: Fix a bug from an error message 62
15.2 Workflow 2: Add a feature with plan mode 63
15.3 Workflow 3: Learn an unfamiliar project 64
15.4 Workflow 4: Tests first 64
15.5 Workflow 5: Ship a pull request 64
15.6 Workflow 6: Long tasks in the background 65
15.7 Workflow 7: Scripts with `claude -p` 65
15.8 Daily routine checklist 65

16 Troubleshooting and FAQ 66

16.1 First-aid routine 66
16.2 Symptom table 67
16.3 Frequently asked questions 67

17 Hooks, Settings, and Environment Reference 69

17.1 Settings files and precedence 69
17.2 Hook events 70
17.3 Worked example: format every file Claude edits 71
17.4 Environment variables 71

Glossary 89

CHAPTER 1

Welcome to Claude Code

Claude Code is an **AI coding agent**. That means it can do more than answer questions: it can open the files in your project, change them, run programs, check its own work, and keep going until a task is done. You stay in charge the whole time.

IN THIS CHAPTER

- What Claude Code is and what it's good for
- The different places you can use it
- What you need before you start

1.1 What is an "agent"?

You have probably used a **chatbot** such as the Claude app or ChatGPT. You type a question and it types back an answer. If it suggests code, *you* copy it, paste it into a file, and run it.

An **agent** closes that loop. When you ask Claude Code to "add a contact form to my website," it will:

1. Look through your project to understand how it's built.
2. Decide which files need to change.
3. Edit those files (asking your permission first, unless you've told it not to).
4. Run commands, for example starting your website, to check that the change works.
5. Tell you what it did, in plain language.

This cycle of *look → plan → act → check* is called the **agentic loop**. It's what makes Claude Code feel less like a search engine and more like a teammate.

1.2 What can you do with it?

You don't need to be a programmer to get value from Claude Code. People use it to:

- **Build things from a description.** "Make me a one-page website for my bakery with our hours and a menu."
- **Understand unfamiliar code.** "Explain what this project does, as if I'm new to coding."
- **Fix problems.** "When I run the app I get this error. Find out why and fix it."
- **Automate chores.** "Rename all these photos by the date they were taken."
- **Work with Git and GitHub.** "Commit my changes with a clear message and open a pull request."
- **Write documents and data files**, not just code: reports, spreadsheets, scripts, and more.

> **TIP**
>
> You can ask Claude Code about itself. Questions like "what can Claude Code do?" or "how do I add an MCP server?" work, because it has its own documentation built in.

1.3 Where Claude Code runs

Claude Code is one product with several "front doors." They share the same brain and the same settings, so what you learn in this manual works everywhere.

SURFACE	WHAT IT LOOKS LIKE	BEST FOR
Terminal (CLI)	A text window where you type	The full feature set; the focus of this manual
Desktop app	The Claude app's **Code** tab	People who prefer buttons and windows over typing commands
VS Code / Cursor extension	A chat panel inside your code editor	Seeing edits side-by-side with your files
JetBrains plugin	Runs inside IntelliJ, PyCharm, etc.	Developers already using those editors
Web	claude.ai/code	Running tasks in the cloud, even from a phone
Slack, GitHub Actions	Mention Claude in a chat or pull request	Teams and automation

CLI stands for *command-line interface*: a program you control by typing text commands instead of clicking.

1.4 What you need

- [ ] A computer running **macOS 13+**, **Windows 10 (1809+)**, or a recent **Linux** (Ubuntu 20.04+, Debian 10+), with at least 4 GB of RAM
- [ ] An internet connection
- [ ] A **paid Claude account**: Pro, Max, Team, or Enterprise, *or* an Anthropic Console account that pays per use
- [ ] A folder to work in (it can be empty, so you don't need an existing project)

> **NOTE**
>
> The free Claude plan does **not** include Claude Code. The cheapest way in is the Pro plan. See the pricing table in Chapter 3.

CHAPTER 2

Terminal Basics in Ten Minutes

The terminal looks intimidating, but you only need a handful of ideas to use Claude Code comfortably. Think of it as a very literal text message conversation with your computer.

IN THIS CHAPTER

- How to open a terminal on Mac, Windows, and Linux
- Folders, paths, and the "current directory"
- The five commands you'll actually use

2.1 Opening a terminal

OPEN YOUR TERMINAL

1. **On a Mac.**
   Press `Cmd` + `Space`, type `Terminal`, and press `Enter`.

2. **On Windows.**
   Press the `Windows` key, type `PowerShell`, and open **Windows PowerShell** (or **Terminal**, which opens PowerShell by default).

3. **On Linux.**
   Press `Ctrl` + `Alt` + `T` on most systems, or search your apps for "Terminal."

You'll see a line ending in something like `%`, `$`, or `>`. That's the **prompt**: the terminal waiting for you to type. When this manual shows a command, type it after the prompt and press `Enter`.

> **TIP**
> On Windows, the prompt starts with `PS C:\` when you're in **PowerShell** and just `C:\` when you're in the older **Command Prompt (CMD)**. Claude Code works in both, but the install commands differ.

2.2 Folders and paths

Everything on your computer lives in **folders** (also called **directories**). A **path** is the address of a folder or file, like `/Users/sam/projects/bakery` on a Mac or `C:\Users\Sam\projects\bakery` on Windows.

The terminal is always "standing in" one folder, called the **current directory** or **working directory**. Claude Code works on the files in whatever folder you start it from. That's how it knows which project you mean.

The `~` symbol is shorthand for your **home folder** (for example `/Users/sam`).

2.3 The five commands you need

COMMAND	WHAT IT DOES	EXAMPLE
`pwd`	Show which folder you're in ("print working directory")	`pwd`
`ls`	List the files here (`dir` in CMD)	`ls`
`cd`	Change into a folder	`cd projects/bakery`
`cd ..`	Go up one folder	`cd ..`
`mkdir`	Make a new folder	`mkdir bakery`

Here's how to make a practice folder and step inside it:

TERMINAL

```
cd ~
mkdir my-first-project
cd my-first-project
pwd
```

> **TIP**
> Press `Tab` while typing a folder name and the terminal finishes it for you. Press the `↑` arrow to bring back the last command you typed.

2.4 Copying and pasting in a terminal

- **Mac:** Cmd + C and Cmd + V, as usual.
- **Windows Terminal / PowerShell:** Ctrl + V pastes; right-click also pastes.
- **Linux:** usually Ctrl + Shift + V.

In a terminal, Ctrl + C usually means **stop the running program**, not copy.

CHAPTER 3

Install and Sign In

Installation takes about two minutes. Anthropic's recommended method is the **native installer**, which also keeps Claude Code up to date automatically.

IN THIS CHAPTER

- Installing on macOS, Linux, and Windows
- Choosing a plan and logging in
- Checking that everything works

3.1 Install with the native installer

INSTALL CLAUDE CODE

1. **Open a terminal.**
   See Chapter 2 if you're not sure how.

2. **Paste the command for your system and press Enter.**
   macOS, Linux, or WSL:

   TERMINAL

   ```
   curl -fsSL https://claude.ai/install.sh | bash
   ```

   Windows PowerShell:

   POWERSHELL

   ```
   irm https://claude.ai/install.ps1 | iex
   ```

   Windows Command Prompt (CMD):

COMMAND PROMPT

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del install.cmd
```

3. **Wait.**
   The installer shows no progress bar while it downloads. That's normal.

4. **Open a *new* terminal window**
   so it picks up the new program.

5. **Check the install.**
   Type `claude --version`. You should see a version number followed by `(Claude Code)`.

> ⚠ **"COMMAND NOT FOUND"?**
> If your terminal says `claude` isn't found or isn't recognized, the install folder isn't on your **PATH** yet. (The PATH is the list of folders your terminal searches for programs.) Close every terminal window and open a fresh one first. If it still fails, run the installer again and read its last few lines. It prints the exact fix.

Other ways to install

METHOD	COMMAND	UPDATES
Homebrew (Mac)	`brew install --cask claude-code`	Manual: `brew upgrade claude-code`
WinGet (Windows)	`winget install Anthropic.ClaudeCode`	Manual: `winget upgrade Anthropic.ClaudeCode`
npm (needs Node.js 22+)	`npm install -g @anthropic-ai/claude-code`	Manual
apt / dnf / apk (Linux)	See the setup page in the official docs	Through your system updates

> ⚠ **DANGER**
> Never install with `sudo npm install -g`. Running npm as an administrator causes permission problems and is a security risk.

Windows notes

- You do **not** need to run the installer as Administrator.
- Installing **Git for Windows** (from git-scm.com) is recommended. It gives Claude Code a Bash shell to run commands with. Without it, Claude Code uses PowerShell instead, which also works.
- If you already use **WSL** (Windows Subsystem for Linux), install from inside your WSL terminal using the macOS/Linux command.

3.2 Choose a plan

Claude Code needs a paid account. Prices below are from claude.com/pricing in October 2026:

PLAN	PRICE	WHO IT'S FOR
Free	$0	Chat only, **no Claude Code**
Pro	$20/month ($17/month billed yearly)	Learning and light daily use
Max 5x	$100/month	Heavy daily use: 5× Pro's usage
Max 20x	$200/month	All-day use: 20× Pro's usage
Team	$25/seat/month Standard, $125 Premium (less yearly)	Small teams, 2–150 people
Enterprise	Custom	Large organizations
Console (API)	Pay per use	Developers who prefer metered billing

> **TIP**
> Start with **Pro**. If you regularly hit your usage limit, upgrade to Max later. Inside Claude Code, `/usage` shows how much of your limit you've used.

3.3 Log in for the first time

SIGN IN

1. **Go to your project folder.**
   For example: `cd ~/my-first-project`.
2. **Start Claude Code.**
   Type `claude` and press `Enter`.
3. **Pick a theme.**
   Use the arrow keys to choose light or dark.

4. **Choose how to log in.**
   Pick your Claude subscription (Pro/Max/Team/Enterprise) or Anthropic Console.
5. **Approve in your browser.**
   A browser window opens; click **Authorize**. Return to the terminal.
6. **Trust the folder.**
   Claude Code asks whether you trust the files in this folder. For your own projects, choose **Yes**.

You only log in once; Claude Code remembers your credentials. To switch accounts later, type `/login` inside Claude Code.

3.4 Health check and updates

- `claude doctor` checks your installation and settings and suggests fixes. Run it whenever something feels off.
- Native installs **update themselves** in the background. To update right now: `claude update`.
- Prefer slower, battle-tested releases? Type `/config` and set **Auto-update channel** to **stable** (about a week behind, skipping releases with major bugs).

CHAPTER 4

Your First Session

Let's build something. In this chapter you'll start Claude Code, ask it questions, and have it create and change real files.

IN THIS CHAPTER

- What the Claude Code screen shows you
- Asking questions and requesting changes
- Approving actions, interrupting, and undoing

4.1 The Claude Code screen

When you run `claude`, you'll see:

- A **welcome panel** showing the version, the AI model in use, and your current folder.
- A **prompt box** where you type, marked with `>`.
- A **status bar** underneath showing the current *permission mode* (more on this in Chapter 6).

Type your request in plain English and press `Enter`. There's no special syntax to learn.

> **TIP**
>
> To write several lines before sending, press `Shift` + `Enter` for a new line (or type `\` then `Enter`, which works in every terminal).

4.2 Build a small website

YOUR FIRST PROJECT

1. **Start in an empty folder.**

   TERMINAL

   ```
   mkdir ~/bakery-site
   cd ~/bakery-site
   claude
   ```

2. **Describe what you want.**

   PROMPT

   Create a simple one-page website for a bakery called "Rise & Shine".
   Include our hours (Tue–Sun, 7am–3pm), a short menu with prices,
   and a contact section. Keep it to plain HTML and CSS.

3. **Watch it plan and work.**
   Claude explains what it's about to do, then creates files such as `index.html` and `styles.css`.

4. **Approve when asked.**
   If Claude asks permission to create a file, choose **Yes**. (Chapter 6 explains how to approve whole categories at once.)

5. **Open the result.**
   Ask: `open the website in my browser`. Or open `index.html` from your file manager.

6. **Change it.**
   Try: `make the colors warmer and add a photo placeholder at the top`.

That's the whole rhythm: **describe, review, refine.**

4.3 Ask questions about existing code

Claude Code shines when you point it at a project someone else wrote. Start it inside the project folder and try:

PROMPT

what does this project do?

PROMPT

explain the folder structure like I'm new to coding

PROMPT

where is the main entry point, and what happens when the app starts?

You don't need to tell Claude which files to read. It looks around by itself. If you *want* to point at a file, type `@` and start typing its name; Claude Code autocompletes the path.

4.4 Approving, interrupting, and undoing

Permission prompts. Before Claude edits a file or runs a command it isn't sure about, it may show what it intends to do and offer choices like:

- **Yes**: allow this one action.
- **Yes, and don't ask again**: allow this kind of action from now on in this project.
- **No**: refuse, and optionally tell Claude what to do instead.

Interrupting. Press `Esc` at any time to stop Claude mid-task. It keeps what it has done so far, and you can redirect it: "Stop. Use blue instead of green."

Undoing. Claude Code saves a **checkpoint** before each change. Press `Esc` twice on an empty prompt (or type `/rewind`) to roll the conversation *and/or* your files back to an earlier point.

> ⚠ WARNING
>
> Checkpoints cover file edits Claude makes. They can't undo things outside your computer, like a sent email, a deleted cloud file, or a database change. For real safety, use Git (Chapter 6 and the GitHub manual in this series).

4.5 Ending and resuming

TO...	DO THIS
Leave Claude Code	Type `/exit`, or press `Ctrl` + `D` twice
Start fresh in the same window	`/clear`
Continue your last conversation	`claude -c` (from the terminal)
Pick an older conversation	`claude -r`, or `/resume` inside Claude Code
Name a session to find it later	`/rename bakery-site`

CHAPTER 5

Talking to Claude Effectively

Claude Code is capable, but it isn't a mind reader. A little care in how you ask makes a huge difference in what you get back.

IN THIS CHAPTER

- The habits that produce great results
- Plan mode: think first, then build
- Sharing screenshots, errors, and files

5.1 Five habits of great prompts

1. **Be specific.** Instead of "fix the bug," say "users see a blank screen after entering a wrong password, so fix that."
2. **Give the goal, not just the task.** "I want customers to order ahead online" helps Claude make better choices than "add a form."
3. **Break big jobs into steps.** Number them, and Claude will work through them in order.
4. **Let Claude explore first.** "Read the code and explain how login works before changing anything."
5. **Say how you'll judge success.** "When you're done, run the tests and show me they pass."

PROMPT

```
1. Add a "Order ahead" page with a form for name, item, and pickup time.
2. Save orders to a file called orders.csv.
3. Add a link to the new page in the site's menu.
4. Open the site and confirm the form works before you finish.
```

5.2 Plan mode: look before you leap

For anything bigger than a small tweak, ask Claude to **plan first**. In **plan mode** Claude researches and writes a step-by-step plan but won't edit your files until you approve it.

- Press Shift + Tab until the status bar says `plan mode on`, **or**
- Start your message with `/plan`, for example: `/plan add online ordering`.

When the plan appears you can approve it, ask for changes ("No, keep planning, and don't use a database"), or press Ctrl + G to edit the plan yourself in a text editor.

> **TIP**
> Plan mode is the single best habit for beginners. Reading the plan teaches you how the project works, and it catches misunderstandings before any files change.

5.3 Share what you see

- **Error messages:** copy and paste them straight into the prompt. Include the whole thing.
- **Screenshots:** paste an image with Ctrl + V (Cmd + V in iTerm2, Alt + V on Windows). Claude can see it.
- **Files:** type `@` followed by part of a file name.
- **A command's output:** start a line with `!` to run a shell command yourself, for example `!npm test`. Claude sees the result and responds to it.

5.4 Ask a side question

Want to ask something without derailing the current task? Use `/btw`:

PROMPT

```
/btw what's the difference between HTML and CSS?
```

The answer appears without being added to the main conversation.

5.5 Keep conversations focused

Claude can hold a lot in mind at once, but that space, the **context window**, is not unlimited. Long, wandering sessions get slower and cost more of your usage.

- Use `/clear` when you switch to an unrelated task.
- Use `/compact` to summarize a long conversation and free up space while staying on the same task.
- Use `/context` to see how full the context window is.

CHAPTER 6

Permissions and Safety

Claude Code can edit files and run commands on your computer, so it comes with a permission system: rules for what it may do without asking. Understanding it lets you work faster *and* safer.

IN THIS CHAPTER

- The six permission modes and how to switch
- Allow, ask, and deny rules
- Safety nets: checkpoints, Git, and the sandbox

6.1 Permission modes

A **permission mode** sets how much Claude can do before checking with you. Press Shift + Tab to cycle modes during a session. The status bar shows the active one.

MODE	RUNS WITHOUT ASKING	GOOD FOR
Manual (`default`)	Reading files only	Sensitive work; learning what Claude does
Accept edits (`acceptEdits`)	Reading, editing files, and simple file commands like `mkdir`, `mv`, `cp`	Iterating on code you'll review afterwards
Plan (`plan`)	Reading and exploring; no edits until you approve a plan	Thinking before building
Auto (`auto`)	Almost everything, with a safety **classifier** checking each action in the background	Long tasks with few interruptions
Don't ask (`dontAsk`)	Only pre-approved tools; everything else is refused	Scripts and automation
Bypass (`bypassPermissions`)	Everything, no checks	Disposable containers and virtual machines **only**

Auto mode is the starting mode for new terminal and VS Code sessions on recent versions of Claude Code. A second AI model (the **classifier**) reviews each risky action, such as running a script from the internet, force-pushing code, deploying, or deleting files that existed before the session, and blocks anything that goes beyond what you asked for.

> ☆ IMPORTANT
>
> You can tell auto mode about limits in plain English: "don't push anything until I've reviewed it." The classifier treats that as a rule for the rest of the conversation. For a hard guarantee, add a **deny rule** (below).

To start a session in a particular mode:

TERMINAL

```
claude --permission-mode plan
```

6.2 Allow, ask, and deny rules

Rules let you pre-approve safe actions and forbid dangerous ones. Type `/permissions` to see and edit them, or write them in a settings file.

- **Allow**: run without asking.
- **Ask**: always ask first, even in auto mode.
- **Deny**: never allowed, in any mode.

Deny rules win over ask rules, which win over allow rules.

.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "allow": [
      "Bash(npm run *)",
      "Bash(git commit *)"
    ],
    "ask": [
      "Bash(git push *)"
    ],
    "deny": [
      "Read(./.env)",
      "Bash(rm -rf *)"
    ]
  }
}
```

The `*` matches anything, so `Bash(npm run *)` covers `npm run build`, `npm run test`, and so on. Put the `*` *after* the sub-command: `Bash(git commit *)` is narrow, but `Bash(git *)` allows every Git command.

> **TIP**
> Tired of approving the same harmless commands? Type `/fewer-permission-prompts`. Claude scans your recent sessions and proposes a safe allow-list for read-only commands you use often.

6.3 Where settings live

Claude Code reads settings from several files. When the same setting appears in more than one place, the higher one in this list wins:

PRIORITY	FILE	APPLIES TO
1 (highest)	Managed settings from your organization	Everyone in your company
2	`claude --settings …` on the command line	This session
3	`.claude/settings.local.json`	You, in this project (not shared)
4	`.claude/settings.json`	Everyone working in this project
5	`~/.claude/settings.json`	You, in every project

`~` means your home folder. A file inside `.claude/` lives in the project folder.

6.4 Safety nets

- **Checkpoints:** `/rewind` (or Esc Esc) restores earlier file versions.
- **Git:** commit your work before big changes so you can always get back. Ask Claude: "commit everything with a clear message before you start."
- **Protected files:** Claude never silently edits things like `.git`, `.bashrc`, or its own `.claude` settings. It always asks, except in bypass mode.
- **Sandbox:** type `/sandbox` (macOS, Linux, WSL2) to run Claude's shell commands in an isolated box with limited file and network access.

> **BYPASS MODE IS FOR THROWAWAY MACHINES**
> `--dangerously-skip-permissions` turns off every check. A bad instruction hidden in a web page or file (a **prompt injection**) could then make Claude do real damage. Only use it inside a container or virtual machine you're happy to wipe. The *Maximum Autonomy* guide in this series covers how to set that up.

CHAPTER 7

Memory: Teaching Claude Your Project

Every session starts with a blank slate. Two features carry knowledge from one session to the next: **CLAUDE.md files**, which you write, and **auto memory**, which Claude writes.

IN THIS CHAPTER

- What a CLAUDE.md file is and where to put it
- Generating one with `/init`
- What belongs in it, with an example

7.1 CLAUDE.md files

A `CLAUDE.md` file is a plain-text note, written in **Markdown** (a simple way to format text with `#` headings and `-` bullets), that Claude reads at the start of every session. Use it for anything you'd otherwise re-explain each time: how to run the project, house style, "always do X" rules.

WHERE	FILE	WHO IT'S FOR
Your whole computer	`~/.claude/CLAUDE.md`	Your personal preferences, every project
A project (shared)	`./CLAUDE.md` or `./.claude/CLAUDE.md`	Everyone on the project, checked into Git
A project (just you)	`./CLAUDE.local.md`	Your private notes; add it to `.gitignore`

7.2 Create one with /init

Inside a project, type:

PROMPT

```
/init
```

Claude studies the project and writes a starter `CLAUDE.md` with build commands, test instructions, and conventions it finds. If one already exists, it suggests improvements instead of overwriting.

7.3 What to put in it

CLAUDE.MD

```
# Rise & Shine bakery website

## How to preview
- Open index.html in a browser. No build step.

## Style
- Plain HTML and CSS only. No frameworks.
- Colors: warm cream (#FFF6E9) and cinnamon (#B5562B).
- Write friendly, short copy. No jargon.

## Rules
- Keep every page working on phones.
- Ask before deleting any image.
```

Tips from Anthropic's guidance:

- **Keep it short:** aim for under about 200 lines. Long files get followed less reliably.
- **Be concrete:** "Use 2-space indentation" beats "format code nicely."
- **Add a line when Claude makes the same mistake twice.**
- **Import other files** with `@`, e.g. `See @README.md for an overview.`

> **NOTE**
>
> CLAUDE.md is *guidance*, not a lock. To truly block an action, use a deny rule (Chapter 6) or a hook (Chapter 9).

7.4 Auto memory

As you work, Claude can note your corrections and preferences ("Sam prefers short commit messages") in its own memory files, which load automatically next time. Type `/memory` to view or edit them, open your CLAUDE.md files, or turn auto memory on or off.

CHAPTER 8

Everyday Commands and Shortcuts

You don't need to memorize these. Type `/` at any time to see a searchable menu of everything available. Here are the ones you'll use most.

8.1 Slash commands

COMMAND	WHAT IT DOES
`/help`	Show help and available commands
`/init`	Create a `CLAUDE.md` for this project
`/clear`	Start a new, empty conversation
`/compact`	Summarize the conversation to free up space
`/context`	Show how full the context window is
`/model`	Switch AI model (e.g. `opus`, `sonnet`, `haiku`)
`/plan`	Enter plan mode, optionally with a task
`/permissions`	View and edit allow / ask / deny rules
`/rewind`	Roll back the conversation and/or files
`/resume`	Reopen an earlier conversation
`/diff`	Review all changes made so far
`/review`	Ask Claude to review your changes for bugs
`/memory`	Edit CLAUDE.md files and auto memory
`/mcp`	Manage connected MCP servers (Chapter 9)
`/usage`	See plan limits and usage (`/cost` works too)

`/config`	Open settings: theme, model, editor mode…
`/doctor`	Diagnose and fix setup problems
`/add-dir`	Let Claude work in an extra folder
`/export`	Save the conversation as text
`/btw`	Ask a side question
`/exit`	Quit

8.2 Keyboard shortcuts

KEYS	ACTION
Esc	Stop Claude mid-task
Esc Esc	Clear your draft; on an empty prompt, open rewind
Shift + Tab	Cycle permission modes
Shift + Enter	New line without sending
↑ / ↓	Scroll through previous prompts
Ctrl + R	Search prompt history
Ctrl + C	Interrupt; on an empty prompt, press twice to exit
Ctrl + O	Show the detailed transcript of what Claude did
Ctrl + G	Edit your prompt (or a plan) in a text editor
Ctrl + V	Paste an image
Ctrl + B	Send a long-running command to the background
?	Show the shortcut help panel (on an empty prompt)

8.3 Starting Claude Code from the terminal

COMMAND	WHAT IT DOES
`claude`	Start an interactive session
`claude "fix the build error"`	Start with a first request
`claude -c`	Continue the most recent conversation here
`claude -r`	Choose an older conversation to resume
`claude -p "summarize README.md"`	Answer once and exit (handy in scripts)
`claude --model opus`	Use a specific model this session
`claude --permission-mode plan`	Start in a specific permission mode
`claude -w new-feature`	Work in an isolated copy of the repo (a Git *worktree*)
`claude update`	Update now
`claude doctor`	Check installation health

CHAPTER 9

Extending Claude Code

Out of the box, Claude Code can work with files, run commands, and search the web. You can extend it in five ways. Each has its own job:

FEATURE	IN ONE SENTENCE	LIVES IN
MCP servers	Connect Claude to outside tools: GitHub, Notion, databases, browsers	`claude mcp add …`
Skills	Save a reusable set of instructions as a `/command`	`.claude/skills/<name>/SKILL.md`
Subagents	Specialist helpers with their own instructions and tools	`.claude/agents/<name>.md`
Hooks	Run your own command automatically when something happens	`"hooks"` in settings.json
Plugins	Bundles of the above you can install in one step	`/plugin`

9.1 MCP: connect to other apps

MCP (Model Context Protocol) is an open standard that lets AI tools plug into other software. An **MCP server** is a small connector that gives Claude new abilities, such as reading your Notion pages or creating GitHub issues. (This series has a whole manual on MCP.)

Most modern services offer a **remote** server: a web address. Add one like this:

TERMINAL

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```

Then type `/mcp` inside Claude Code, select the server, and sign in through your browser.

Some servers run as a program on your own computer (a **local** or **stdio** server). Everything after the `--` is the command that starts it:

TERMINAL

```
claude mcp add --env API_KEY=your-key myserver -- npx -y @example/mcp-server
```

COMMAND	WHAT IT DOES
`claude mcp list`	List servers and whether they're connected
`claude mcp get <name>`	Show details for one server
`claude mcp remove <name>`	Remove a server
`--scope user`	Make a server available in all your projects
`--scope project`	Save it in `.mcp.json` to share with your team

> ⚠ **WARNING**
> Only install MCP servers you trust. A server can see what Claude sends it and can return text that tries to trick Claude. Prefer official servers published by the company whose product you're connecting.

9.2 Skills: your own slash commands

A **skill** is a folder containing a `SKILL.md` file of instructions. Claude uses it automatically when it fits the situation, or you run it by name.

~/.CLAUDE/SKILLS/WEEKLY-UPDATE/SKILL.MD

```
---
description: Writes a friendly weekly progress update from recent Git commits. Use when asked for a
status update or weekly summary.
---

Look at the Git commits from the last 7 days. Write a short update with:
- three bullet points of what got done
- one bullet of what's next
Use plain language a non-programmer would understand.
```

Now `/weekly-update` works in every project. Put the folder in a project's `.claude/skills/` instead to share it with your team.

9.3 Subagents: specialist helpers

A **subagent** is a helper Claude can hand work to. Each has its own instructions, its own allowed tools, and its own fresh context, so a big research job doesn't clutter your main conversation. The easiest way to make one is to ask:

PROMPT

Create a project subagent called "copy-editor" that checks my website text for spelling, tone, and reading level. It should only be able to read files.

Claude saves it as `.claude/agents/copy-editor.md`. Claude Code also ships with built-in subagents, such as **Explore** (fast, read-only searching) and **Plan**.

9.4 Hooks: automatic actions

A **hook** runs a command of yours at a specific moment, for example after every file edit or when Claude needs your attention. Unlike CLAUDE.md, hooks *always* run. This example shows a desktop notification on a Mac whenever Claude is waiting for you:

~/.CLAUDE/SETTINGS.JSON

```
{
  "hooks": {
    "Notification": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "osascript -e 'display notification \"Claude needs you\" with title \"Claude
Code\"'"
          }
        ]
      }
    ]
  }
}
```

Type `/hooks` to see what's configured. You can also just describe the hook you want and ask Claude to set it up.

9.5 Plugins: install bundles

A **plugin** packages skills, subagents, hooks, and MCP servers together. Type `/plugin` to browse and install from **marketplaces** (catalogs of plugins). Anthropic's official marketplace is built in.

CHAPTER 10

Beyond the Terminal

Everything you've learned carries over to Claude Code's other surfaces.

10.1 The desktop app

Download the Claude app from claude.com/download, sign in, and open the **Code** tab. You get the same Claude Code with a graphical interface: a folder picker, a visual **diff** (a side-by-side view of what changed), a permission-mode selector next to the send button, and the ability to run several sessions in parallel.

10.2 VS Code and Cursor

In VS Code or Cursor, open the Extensions view (`Ctrl`/`Cmd` + `Shift` + `X`), search for **Claude Code**, and click **Install**. A Claude panel appears in the sidebar. You can review proposed changes as inline diffs and @-mention the lines you've selected.

10.3 Claude Code on the web and phone

At claude.ai/code, or in the **Code** tab of the Claude mobile app, you can start a **cloud session**: Claude works on a copy of your GitHub repository on Anthropic's computers, so it keeps going even when your laptop is closed. When it's done, it pushes a branch you can review. Cloud sessions are available on Pro, Max, Team, and eligible Enterprise seats, and need your GitHub account connected.

From the terminal, `claude --cloud "fix the login bug"` hands a task to the cloud, and `claude --teleport` pulls a cloud session back to your computer.

10.4 Scripts and automation

`claude -p` (“print mode”) answers once and exits, which makes Claude Code usable inside scripts:

TERMINAL

```
cat error.log | claude -p "explain this error in one paragraph"
```

For GitHub, type `/install-github-app` to let your team mention `@claude` in issues and pull requests to get work done automatically.

CHAPTER 11

Costs and Limits

11.1 How usage works

On a **subscription** (Pro, Max, Team), Claude Code use is included up to your plan's limits, which reset over time. On a **Console** account you pay per **token** (a token is a chunk of text, roughly ¾ of a word). Longer conversations and bigger files use more tokens.

Type `/usage` to see your current usage, your limits, and a breakdown of what's using the most.

11.2 Make your usage go further

- `/clear` **between unrelated tasks.** Old conversation history is re-read with every message.
- `/compact` **during long tasks** to shrink the history.
- **Use a lighter model for simple jobs:** `/model haiku` or `/model sonnet` for routine work, and save the most powerful model for hard problems.
- **Be specific.** A precise request avoids wasted exploration.
- **Keep CLAUDE.md lean.** It's loaded into every session.

> **TIP**
>
> If you hit a limit mid-task, `/rate-limit-options` shows your choices: wait for the reset, add usage credits, or upgrade.

CHAPTER 12

Troubleshooting

PROBLEM	TRY THIS
`claude: command not found`	Open a new terminal window. Still failing? Re-run the installer and follow the PATH fix it prints, or run `claude doctor`.
`'irm' is not recognized`	You're in CMD, not PowerShell. Use the CMD install command, or open PowerShell.
`The token '&&' is not a valid statement separator`	You're in PowerShell. Use the PowerShell command instead.
Login window doesn't open	Copy the link Claude Code prints into your browser manually.
Claude keeps asking permission for the same thing	Choose **Yes, and don't ask again**, add an allow rule, or run `/fewer-permission-prompts`.
Claude ignores an instruction	Put it in CLAUDE.md, make it concrete, and keep the file short. For must-never-happen rules, use a deny rule.
Responses getting slow or confused	`/compact` or `/clear`, then restate the goal.
Something is broken and you don't know why	`/doctor` inside Claude Code. It diagnoses and offers fixes.
MCP server shows "Failed to connect"	`claude mcp get <name>` shows the error; re-authenticate in `/mcp`.

> **NOTE**
>
> Stuck? Type `/bug` to report a problem to Anthropic, ask Claude itself ("why did that fail?"), or take the free Claude Code courses at academy.claude.com.

Quick Reference

Install (Mac/Linux): `curl -fsSL https://claude.ai/install.sh | bash`
Install (Windows PowerShell): `irm https://claude.ai/install.ps1 | iex`
Start: `cd my-project` then `claude` · **Continue:** `claude -c` · **Resume:** `claude -r`

Stop: Esc · **Undo:** Esc Esc or `/rewind` · **Modes:** Shift + Tab · **New line:** Shift + Enter

Plan first: `/plan …` · **Fresh start:** `/clear` · **Shrink history:** `/compact` · **Project memory:** `/init`

Connect an app: `claude mcp add --transport http <name> <url>` then `/mcp` to sign in

Health check: `claude doctor` · **Update:** `claude update` · **Usage:** `/usage`

CHAPTER 13

The Complete Slash-Command Reference

Slash commands are the control panel of Claude Code. You type `/` at the start of a message and a searchable menu appears. This chapter lists **every built-in command and bundled skill documented by Anthropic** as of this edition, so you can look any of them up on paper.

IN THIS CHAPTER

- How slash commands, bundled skills, and your own skills differ
- Deep dives on the commands you'll use every day: `/clear`, `/compact`, `/context`, `/rewind`, `/resume`, `/model`, `/init`, `/memory`, `/permissions`
- A full A–Z table of every documented command
- Recipes that chain commands together

WHERE THIS LIST COMES FROM

The A–Z table was generated from Anthropic's official commands page, code.claude.com/docs/en/commands, in October 2026. Claude Code updates often. Some commands appear only on certain plans, platforms, or versions. If a command here doesn't show up for you, run `claude update`, then type `/` and search for it. Your own `/` menu is always the final word.

13.1 Three kinds of slash command

KIND	WHAT IT IS	EXAMPLE
Built-in command	Fixed logic inside Claude Code. It runs instantly and usually doesn't cost tokens.	`/clear`, `/model`, `/exit`
Bundled skill	A ready-made set of instructions that Claude follows using its tools. It uses tokens, like a normal request.	`/code-review`, `/debug`, `/batch`
Your own skill	A `SKILL.md` file you or your team wrote (Chapter 9).	`/deploy`, `/fix-issue 123`

Anything you type after the command name is passed in as **arguments**. For example, in `/compact keep the API decisions`, the text after `/compact` tells the summary what to focus on.

> **TIP**
> A command only runs when it's at the **start** of your message. If you write "please /deploy this" in the middle of a sentence, nothing runs on its own. Claude just treats it as permission to use that skill if it decides to.

13.2 /clear: start fresh

`/clear` wipes the conversation and starts a new one with empty context. Your files are **not** touched. Only Claude's short-term memory of the chat is reset. `CLAUDE.md` loads again automatically.

USE /CLEAR BETWEEN TASKS

1. **Finish or commit the current task.**
   Ask Claude to summarize what it did if you want a record.
2. **Type `/clear`.**
   Optionally add a name, such as `/clear login-work`, to label the old conversation so you can find it later.
3. **Start the next task.**
   Claude now has a clean, fast, cheap context.
4. **Need the old chat back?**
   Use `/resume` and pick it from the list.

Use `/clear` when you switch to an unrelated task, when Claude keeps repeating an old mistake, or when the conversation has drifted. `/reset` and `/new` are aliases.

13.3 /compact: shrink without forgetting

Every message, file, and command output takes up room in the **context window**, the AI's working memory. When it fills up, answers get slower, more expensive, and less sharp. `/compact` replaces the conversation so far with a summary, so you keep the important facts and free most of the space.

PROMPT

/compact Keep the database schema decisions and the list of failing tests. Drop the exploration of old files.

SITUATION	USE
Same task, but the chat is long	`/compact`
New, unrelated task	`/clear`
Claude went down a wrong path a few steps ago	`/rewind`
You want a side answer without cluttering the chat	`/btw`

Claude Code also **auto-compacts** when the window is nearly full. `/autocompact` lets you set how full it gets before that happens. Pass a size such as `500k`, or `auto` to go back to the default.

> ⚠ SUMMARIES LOSE DETAIL
>
> Compaction is lossy. Exact error messages, file paths, and small decisions can disappear. Before compacting, put anything that must survive into `CLAUDE.md` or a notes file, or list it in your `/compact` instructions.

13.4 /context: see what's using space

`/context` draws a coloured grid of your context window, broken down by system prompt, tools, MCP servers, memory files, skills, and messages. It also suggests what to trim. Check it whenever Claude feels slow or forgetful. If MCP tools take up a big block, disconnect the servers you aren't using with `/mcp`.

13.5 /rewind: the undo button

`/rewind` (aliases `/checkpoint` and `/undo`) rolls the conversation, your code, or both back to an earlier message. Pressing `Esc` twice on an empty prompt opens the same menu.

UNDO A BAD CHANGE

1. **Type `/rewind`.**
   A list of your earlier messages appears.
2. **Pick the last good point.**
   Choose the message *before* things went wrong.
3. **Choose what to restore.**
   Code only, conversation only, or both.
4. **Re-ask, more precisely.**
   Add the detail that was missing the first time.

> **DANGER**
> Checkpoints track edits Claude made with its file tools. Changes made by shell commands, such as `rm` or a database migration, and edits from background forked skills are **not** undone. Git is your real safety net, so commit often.

13.6 /resume, /branch, /fork, and /rename

COMMAND	USE IT TO…
`/resume`	Reopen an earlier conversation by name or from a picker
`/rename my-feature`	Give this session a memorable name
`/branch`	Split the conversation here so you can try another direction, keeping the original
`/fork`	Copy the conversation into a new background session and keep working here

13.7 /model and /effort

`/model` switches which Claude model you're using and saves it as the default for new sessions. `/effort` sets how hard the model thinks: `low`, `medium`, `high`, `xhigh`, `max`, or `auto`. Higher effort is slower and uses more of your plan, but it's better for tricky bugs and design work. A good habit is to plan with a strong model at high effort, then do routine edits at a lower setting.

13.8 /init and /memory

`/init` reads your project and writes a starter `CLAUDE.md`. `/memory` opens your `CLAUDE.md` files for editing and lets you turn **auto memory** on or off and browse what it has saved. Run `/init` once per project, then keep the file current with `/memory`.

13.9 /permissions, /sandbox, and /fewer-permission-prompts

`/permissions` shows and edits your allow, ask, and deny rules (Chapter 6). `/sandbox` toggles sandbox mode on supported systems. The bundled `/fewer-permission-prompts` skill scans your history for safe, read-only commands you approve often, then suggests allow rules for them. Review that list before you accept it.

13.10 Review and quality commands

COMMAND	WHAT IT'S FOR
`/diff`	See every change in your working tree
`/code-review` (alias `/review`)	Have Claude review your diff, a branch, or a PR for bugs
`/security-review`	Scan your branch's changes for security risks
`/simplify`	Run parallel reviewers that tidy up changed code
`/verify` and `/run`	Build and run your app to confirm a change actually works

13.11 Housekeeping commands

`/doctor` runs a setup checkup and offers fixes. `/usage` (aliases `/cost` and `/stats`) shows plan limits and spending. `/status` shows your version, model, and account. `/config` opens settings. `/export` saves the chat as text, `/copy` copies the last answer, and `/release-notes` shows what's new. `/exit` (or `/quit`) leaves.

13.12 Recipes: commands that work well together

Long feature, keep it sharp: `/plan` → approve → work → `/compact keep decisions` → continue → `/code-review` → commit.

Switching tasks: commit → `/clear new-task` → `/context` to confirm it's empty.

Undo a wrong turn: `/rewind` → restore code and conversation → re-prompt with more detail.

Before a pull request: `/diff` → `/code-review` → `/security-review` → `/verify`.

Something's broken in Claude Code itself: `/doctor` → `/debug describe the problem` → `/bug` if it's a real bug.

Running low on plan usage: `/usage` → `/model` to a lighter model → `/effort low` for routine edits.

13.13 Every documented command, A–Z

The **Kind** column shows whether the command is built in, a bundled skill (uses tokens as Claude works), or a workflow. Square brackets `[ ]` mean the argument is optional. Angle brackets `< >` mean it's required.

COMMAND	KIND	WHAT IT DOES
`/add-dir <path>`	Built-in	Add a working directory for file access during the current session.
`/advisor [model|off]`	Built-in	Enable or disable the advisor tool, which consults a second model for guidance at key moments during a task.
`/agents`	Built-in	Print a reminder to ask Claude to create or manage subagents, or to edit `.claude/agents/` or `~/.claude/agents/` directly.
`/artifact-capabilities`	Skill	Load the reference for the runtime capabilities a published artifact can use, such as calling your connectors or offering a file download, including which ones your account has.
`/artifact-diagramming`	Skill	Load diagramming guidance for Claude to follow in artifacts: when a diagram helps, what to draw, and how to write inline SVG that stays legible in light and dark themes.

`/artifacts`	Built-in	List the artifacts you own or that are shared with you, then attach one to the session, open it in your browser, or copy its link.
`/auto-mode-setup`	Built-in	Draft `autoMode.environment` entries from your project and recent sessions, then review the draft and save it to your user settings.
`/autocompact [auto|<tokens>]`	Built-in	Set the auto-compact window: how full the context window gets before Claude Code compacts automatically.
`/autofix-pr [prompt]`	Built-in	Spawn a cloud session that watches the current branch's PR and pushes fixes when CI fails or reviewers leave comments.
`/background [prompt]`	Built-in	Detach the current session to run as a background agent and free this terminal. Alias: `/bg`.
`/batch <instruction>`	Skill	Orchestrate large-scale changes across a codebase in parallel.
`/branch [name]`	Built-in	Create a branch of the current conversation at this point, so you can try a different direction without losing the conversation as it stands.
`/btw [question]`	Built-in	Ask a side question about the current session without adding to the conversation.
`/bug [report]`	Built-in	Report a bug or share your conversation. Alias: `/share`.
`/cd <path>`	Built-in	Move this session to a new working directory, keeping the conversation.
`/chrome`	Built-in	Configure Claude in Chrome settings.
`/claude-api [migrate|upgrade|managed-agents-onboard|prompt-audit|cost-optimize|build-eval|hillclimb|preserved-thinking-migration]`	Skill	Load Claude API and Managed Agents reference material for your project's language.
`/claude-in-chrome [task]`	Skill	Have Claude carry out a task in your browser, such as testing a page, filling a form, or reading console logs, through Claude in Chrome.
`/clear [name]`	Built-in	Start a new conversation with empty context. Alias: `/reset`, `/new`.
`/code-review [low|medium|high|xhigh|max|ultra] [--fix] [--comment] [--max-findings n|all|default] [pr#|branch|path]`	Skill	Review the current diff, or a PR number, branch, or path you pass, for correctness bugs. Alias: `/review`.
`/color [color|default]`	Built-in	Set the prompt bar color for the current session.
`/compact [instructions]`	Built-in	Free up context by summarizing the conversation so far.
`/config [key=value ...]`	Built-in	Open the Settings interface to adjust theme, model, output style, and other preferences. Alias: `/settings`.
`/context [all]`	Built-in	Visualize current context usage as a colored grid.

`/copy [N]`	Built-in	Copy the last assistant response to clipboard.
`/cost`	Built-in	Alias for `/usage`.
`/dataviz [request]`	Skill	Design guidance for charts, graphs, and dashboards.
`/debug [description]`	Skill	Enable debug logging for the current session and troubleshoot issues by reading the session debug log.
`/deep-research <question>`	Workflow	Fan out web searches on a question, fetch and cross-check sources, and synthesize a cited report.
`/design [brief]`	Skill	Draft UI mockups, screen flows, landing pages, or posters as artboards on one canvas, published as a Claude Design artifact, for example `/design a settings screen for a mobile banking app`.
`/design-login`	Built-in	Authorize design-system access for `/design-sync` with your claude.ai account.
`/design-sync [hint]`	Skill	Convert your repo's React design system and upload it to Claude Design, so designs it produces use your real components.
`/desktop`	Built-in	Continue the current session in the Claude Code Desktop app. Alias: `/app`.
`/diff`	Built-in	Review the changes in your working tree, including the edits Claude has made so far.
`/doctor [prompt-audit [path]]`	Skill	Run a setup checkup that diagnoses installation, settings, extension, and `CLAUDE.md` problems and proposes fixes that Claude applies after you confirm. Alias: `/checkup`.
`/effort [level|auto|status|ultracode [on|off]]`	Built-in	Set the effort level: `low` to `xhigh`, `max`, or `auto`; `status` prints it. `ultracode` or `ultracode on` turns ultracode on for the session at the current level, and `ultracode off` turns it off; the `ultracode` key...
`/exit`	Built-in	Exit the CLI. In an attached background session, this detaches and the session keeps running. Alias: `/quit`.
`/export [filename]`	Built-in	Export the current conversation as plain text.
`/fast [on|off]`	Built-in	Toggle fast mode on or off.
`/feedback [report]`	Built-in	Send product feedback about Claude Code.
`/fewer-permission-prompts`	Skill	Scan your transcripts for common read-only Bash and MCP tool calls, then add a prioritized allowlist to project `.claude/settings.json` to reduce permission prompts.
`/focus`	Built-in	Toggle the focus view, which shows only your last prompt, a one-line tool-call summary with edit diffstats, and the final response.
`/fork [prompt]`	Built-in	Copy the current conversation into a new background session and keep working here.
`/goal [condition|clear]`	Built-in	Set a goal: Claude keeps working across turns until the condition is met or the goal clears for another reason.
`/heapdump`	Built-in	Write a JavaScript heap snapshot and a memory breakdown to `~/Desktop`, or your home directory on Linux without a Desktop folder, for diagnosing high memory usage.
`/help`	Built-in	Show help and available commands.
`/hooks`	Built-in	View hook configurations.
`/ide`	Built-in	Manage IDE integrations and show status.

`/import [codex|gemini|cursor] [--dry-run] [--yes]`	Built-in	Bring configuration from OpenAI Codex, Google Gemini CLI, or Cursor on your machine into Claude Code, including instruction files, MCP servers, commands, subagents, and skills.
`/init`	Built-in	Initialize project with a `CLAUDE.md` guide.
`/insights`	Built-in	Generate an HTML report analyzing your recent sessions on this machine: which projects you work in, how you use Claude Code, where things go wrong, and features to try.
`/install-github-app`	Built-in	Install the Claude GitHub App for a repository, with an optional step to set up GitHub Actions workflows and secrets.
`/install-slack-app`	Built-in	Install the Claude Slack app.
`/keybindings`	Built-in	Open your keyboard shortcuts file.
`/list-agents`	Built-in	List the subagents, agent team teammates, and other Claude Code sessions Claude can message, with the name to use for each.
`/login`	Built-in	Sign in to your Anthropic account.
`/logout`	Built-in	Sign out from your Anthropic account.
`/loop [interval] [prompt]`	Skill	Run a prompt repeatedly while the session stays open. Alias: `/proactive`.
`/mcp [reconnect (<server>|all)|enable|disable [<server>|all]]`	Built-in	Manage MCP server connections and OAuth authentication.
`/memory`	Built-in	Edit `CLAUDE.md` files, enable or disable auto memory, and view auto memory entries.
`/mobile`	Built-in	Show QR code to download the Claude mobile app. Alias: `/ios`, `/android`.
`/model [model]`	Built-in	Switch the AI model and save it as your default for new sessions.
`/output-style [style]`	Built-in	List output styles or switch to one, for example `/output-style concise`.
`/passes`	Built-in	Share a free week of Claude Code with friends.
`/permissions`	Built-in	Manage allow, ask, and deny rules for tool permissions. Alias: `/allowed-tools`.
`/plan [description]`	Built-in	Enter plan mode directly from the prompt.
`/plugin [subcommand]`	Built-in	Manage Claude Code plugins.
`/plugin-authoring`	Built-in	Load the reference Claude works from to write a mod.
`/powerup`	Built-in	Discover Claude Code features through quick interactive lessons with animated demos.

`/pr-comments [PR]`	Built-in	Removed in v2.1.91. Ask Claude directly to view pull request comments instead.
`/privacy-settings`	Built-in	View and update your privacy settings.
`/radio`	Built-in	Open Claude FM lo-fi radio in your browser.
`/rate-limit-options`	Built-in	Show ways to keep working when a claude.ai usage limit blocks a request: wait and continue automatically when the limit resets, add usage credits, or upgrade your plan.
`/recap`	Built-in	Generate a one-line summary of the current session on demand.
`/release-notes`	Built-in	View the changelog in an interactive version picker.
`/reload-plugins [--force]`	Built-in	Reload all active plugins to apply pending changes without restarting.
`/reload-skills`	Built-in	Re-scan skill and command directories so skills added or changed on disk during the session become available without restarting.
`/remote-control`	Built-in	Make this session available for Remote Control from claude.ai. Alias: `/rc`.
`/remote-env`	Built-in	Choose the default cloud environment for cloud sessions you start from the CLI.
`/rename [name]`	Built-in	Rename the current session and show the name on the prompt bar.
`/resume [session]`	Built-in	Resume a conversation by ID or name, or open the session picker. Alias: `/continue`.
`/review [low|medium|high|xhigh|max|ultra] [--fix] [--comment] [--max-findings n|all|default] [pr#|branch|path]`	Built-in	Alias of `/code-review`: reviews the current diff, or a PR number, branch, or path you pass, such as `/review 1234`, and takes the same effort levels and flags.
`/rewind`	Built-in	Rewind the conversation and/or code to a previous point, or summarize from a selected message. Alias: `/checkpoint`, `/undo`.
`/run`	Skill	Launch and drive your project's app to see a change working, not only passing tests.
`/run-skill-generator`	Skill	Teach `/run` and `/verify` how to build, launch, and drive your project's app from a clean environment by writing a per-project skill.
`/sandbox`	Built-in	Toggle sandbox mode.
`/schedule [description]`	Built-in	Create, update, list, or run routines, which execute in the cloud. Alias: `/routines`.
`/scroll-speed`	Built-in	Adjust mouse wheel scroll speed interactively.
`/security-review`	Built-in	Analyze the changes on your current branch for security vulnerabilities.

`/setup-bedrock`	Built-in	Configure Amazon Bedrock authentication, region, and model pins through an interactive wizard.
`/setup-vertex`	Built-in	Configure Google Cloud's Agent Platform authentication, project, region, and model pins through an interactive wizard.
`/simplify [target]`	Skill	Review the changed code for cleanup opportunities and apply the fixes.
`/skill-doctor`	Built-in	Show what each of your skills costs in context and how often it gets used, so you can find skills to turn off.
`/skills`	Built-in	List available skills.
`/slides [brief]`	Skill	Make a new presentation as a Claude Slides artifact filled from your brief, for example `/slides a quarterly review of the platform team`.
`/stats`	Built-in	Alias for `/usage`.
`/status`	Built-in	Open the Settings interface on the Status tab, showing version, model, account, and connectivity.
`/statusline`	Built-in	Configure Claude Code's status line.
`/stickers`	Built-in	Order Claude Code stickers.
`/stop`	Built-in	Stop the background session you're attached to, or the one you send it to as a peek reply; the transcript and any worktree are kept.
`/subtask <task>`	Built-in	Spawn a forked subagent: a background subagent that inherits the full conversation and works on the task while you keep working.
`/tasks`	Built-in	View and manage background work in the current session, including subagents that have finished.
`/team-onboarding`	Built-in	Generate a team onboarding guide from your Claude Code usage history.
`/teleport`	Built-in	Pull a cloud session into this terminal.
`/terminal-setup`	Built-in	Install a Shift+Enter keybinding for newlines in VS Code, Cursor, Devin Desktop, Alacritty, or Zed.
`/theme`	Built-in	Change the color theme.
`/tui [default|fullscreen]`	Built-in	Set the terminal UI renderer and relaunch into it with your conversation intact. `fullscreen` enables the flicker-free alt-screen renderer.
`/ultraplan <prompt>`	Built-in	Removed.
`/ultrareview [PR or branch]`	Built-in	Run a deep, multi-agent code review in a cloud sandbox with ultrareview.
`/update-config [request]`	Skill	Describe a settings change, such as allowing a command, setting an environment variable, or adding a hook, and Claude edits the matching `settings.json` file.
`/upgrade`	Built-in	Open the upgrade page in your browser to switch to a higher plan tier.
`/usage`	Built-in	Show session cost, plan usage limits, and activity stats.

`/usage-credits`	Built-in	Configure usage credits, or request them from your admin, when you hit a limit.
`/verify`	Skill	Confirm a code change does what it should by building your project's app, running it, and observing the result, rather than relying on tests or type checks.
`/vim`	Built-in	Removed in v2.1.92. To toggle between Vim and Normal editing modes, use `/config` → Editor mode.
`/voice [hold|tap|off]`	Built-in	Toggle voice dictation, or enable it in a specific mode.
`/web-setup`	Built-in	Connect your GitHub account for cloud sessions using your local `gh` CLI credentials.
`/workflow-authoring`	Skill	Load the reference for writing dynamic workflow scripts: the script API, resume behavior, quality patterns, and worked examples.
`/workflows`	Built-in	Open the workflow progress view to watch, pause, resume, or save running and completed workflows.

NOTE

A few rows say "Removed." They're kept here because older tutorials still mention them. For example, `/vim` was replaced by **Editor mode** in `/config`.

CHAPTER 14

Command-Line Flags and Shortcuts Reference

This chapter is the companion to the slash-command reference. It covers what you can type **in your terminal before** Claude Code starts, and every keyboard shortcut you can press **inside** it. The tables were generated from Anthropic's CLI reference (code.claude.com/docs/en/cli-reference) and interactive-mode page (code.claude.com/docs/en/interactive-mode) in October 2026.

IN THIS CHAPTER

- Every `claude …` subcommand
- Every startup flag, with the ones beginners need marked below
- Every keyboard shortcut and line-editing key

14.1 The ten flags beginners actually use

FLAG	WHY YOU'D USE IT
`-c` / `--continue`	Pick up exactly where you left off in this folder
`-r` / `--resume`	Choose an older conversation
`-p` / `--print`	Ask one question, print the answer, exit, which is great for scripts
`--model`	Pick the model for this session only
`--permission-mode plan`	Start in plan mode, so nothing gets edited until you approve
`--add-dir ../other`	Let Claude also work in a second folder
`--effort high`	Ask for deeper thinking this session
`--debug`	Write a detailed log when something misbehaves
`--chrome`	Turn on browser control for web testing
`--dangerously-skip-permissions`	Skip **all** prompts. Only use it in a throwaway container or VM (see Vol. 8)

> **DANGER**
>
> Flags with "dangerously" in their name remove safety checks. On your everyday computer, use plan mode, accept-edits mode, or allow rules instead.

14.2 Terminal subcommands

COMMAND	WHAT IT DOES
`claude`	Start interactive session.
`claude "query"`	Start interactive session with initial prompt.
`claude -p "query"`	Query via SDK, then exit.
`claude -c`	Continue most recent conversation in current directory.
`claude -c -p "query"`	Continue via SDK.
`claude -r "<session>" "query"`	Resume session by ID or name.
`claude update`	Update to latest version.
`claude gateway`	Start the self-hosted Claude apps gateway server, for administrators deploying SSO and policy in front of Claude Code on Amazon Bedrock, Google Cloud's Agent Platform, or Microsoft Foundry.
`claude install [version]`	Install or reinstall the native binary.
`claude auth login`	Sign in to your Anthropic account.

`claude auth logout`	Log out from your Anthropic account.
`claude auth status`	Show authentication status as JSON. Use `--text` for human-readable output.
`claude agents`	Open agent view to monitor and dispatch parallel background sessions.
`claude attach <id|name>`	Attach to a background session in this terminal.
`claude auto-mode defaults`	Print the built-in auto mode classifier rules as JSON. Use `claude auto-mode config` to see your effective config with settings applied. `--label <prefix>` prints only the rules whose label starts...
`claude auto-mode reset`	Restore the default auto mode configuration by removing the `autoMode` section from your user settings file.
`claude daemon logs`	Follow the background-session supervisor's log file, `~/.claude/daemon.log`, printing new lines as they arrive until you press `Ctrl+C`.
`claude daemon run`	Run the background-session supervisor in the foreground of this terminal, printing its log.
`claude daemon status`	Print the background-session supervisor's state, version, socket directory, and worker count for diagnostics.
`claude daemon stop --any`	Stop the background-session supervisor and the sessions it hosts.
`claude doctor`	Print read-only installation and settings diagnostics from the terminal without starting a session, including install health, settings-file validation errors, and Remote Control eligibility.
`claude import [source]`	Start an interactive session that runs `/import` to bring configuration from other coding agents into Claude Code.
`claude logs <id|name>`	Print recent output from a background session.
`claude mcp`	Configure Model Context Protocol (MCP) servers.
`claude mcp login <name>`	Run a configured MCP server's OAuth flow without opening the interactive `/mcp` panel.
`claude mcp logout <name>`	Clear stored OAuth credentials for an MCP server.
`claude plugin`	Manage Claude Code plugins.
`claude purge [path]`	Delete all local Claude Code state for a project: transcripts, task lists, debug logs, file-edit history, prompt history lines, and the project's entry in `~/.claude.json`.
`claude remote-control`	Start a Remote Control server to control Claude Code from Claude.ai or the Claude app.
`claude respawn <id>`	Restart a background session, running or stopped, with its conversation intact.
`claude rm <id>`	Remove a background session from the list.
`claude self-hosted-runner`	Start a runner process that registers this machine or container with a self-hosted environment and hosts Claude Code cloud sessions on your infrastructure.
`claude setup-token`	Generate a long-lived OAuth token for CI and scripts.
`claude stop <id>`	Stop a background session.
`claude ultrareview [target]`	Run ultrareview non-interactively.

14.3 Every startup flag

FLAG	WHAT IT DOES
`--add-dir`	Add additional working directories for Claude to read and edit files.
`--advisor <model>`	Enable the server-side advisor tool for this session with a model alias, `fable`, `opus`, or `sonnet`, or a full model ID. Takes precedence over the `advisorModel`...
`--agent`	Specify an agent for the current session (overrides the `agent` setting).
`--agents`	Define custom subagents dynamically via JSON. Accepts the fields listed for CLI-defined subagents.
`--allow-dangerously-skip-permissions`	Add `bypassPermissions` to the `Shift+Tab` mode cycle without starting in it.
`--allowedTools`, `--allowed-tools`	Tools that execute without prompting for permission, apart from reads from network paths.
`--append-subagent-system-prompt`	Append custom text to the end of every subagent's system prompt, nested subagents included, apart from a forked subagent, which reuses the conversation's own prompt.
`--append-subagent-system-prompt-file`	Load text from a file and append it to subagent system prompts.
`--append-system-prompt`	Append custom text to the end of the default system prompt.
`--append-system-prompt-file`	Load additional system prompt text from a file and append to the default prompt.
`--autocompact <auto|tokens>`	Set the auto-compact window for this session without changing your saved settings.
`--ax-screen-reader`	Render screen-reader friendly output: flat text without decorative borders or animations.
`--bare`	Minimal mode: skip auto-discovery of hooks, skills, custom commands, subagents, installed plugins, MCP servers, auto memory, and CLAUDE.md so scripted calls start faster.
`--betas`	Beta headers to include in API requests (API key users only).
`--bg`, `--background`	Start the session as a background agent and return immediately.
`--channels`	(Research preview) MCP servers whose channel notifications Claude should listen for in this session.
`--chrome`	Enable Chrome browser integration for web automation and testing.
`--cloud`	With a task description, create a new cloud session.
`--continue`, `-c`	Load the most recent conversation in the current directory, including a background session that has finished; opening finished background sessions requires Claude...
`--dangerously-load-development-channels`	Enable channels that are not on the approved allowlist, for local development.
`--dangerously-skip-permissions`	Skip permission prompts.
`--debug`	Enable debug mode with optional category filtering, such as `--debug='mcp,startup'` or `--debug='!1p'`.
`--debug-file <path>`	Write debug logs to a specific file path.

`--desktop`	Open the Claude Desktop app on the current directory and exit without starting a session in the terminal.
`--disable-slash-commands`	Disable all skills and commands for this session.
`--disallowedTools`, `--disallowed-tools`	Deny rules.
`--effort`	Set the effort level for the current session.
`--enable-auto-mode`	Removed in v2.1.111. Auto mode is now in the `Shift+Tab` cycle by default; use `--permission-mode auto` to start in it.
`--environment <environment-id>`	Create a new cloud session that runs on the self-hosted environment with the given ID. Environment IDs start with `ccpool_`.
`--exclude-dynamic-system-prompt-sections`	Move per-user context, such as the auto memory location, out of the system prompt and into the first user message.
`--exec`	Run a shell command as a PTY-backed background job instead of starting a Claude session.
`--fallback-model`	Enable automatic fallback to the specified model(s) when the primary model is overloaded or not available, for example a retired model.
`--fork-session`	When resuming, create a new session ID instead of reusing the original (use with `--resume` or `--continue`).
`--forward-subagent-text`	Emit subagent text and thinking blocks in the output stream as `assistant` and `user` messages with `parent_tool_use_id` set, so you can reconstruct each subagent's...
`--from-pr`	Open the session picker filtered to sessions linked to a specific pull request.
`--ide`	Automatically connect to IDE on startup if exactly one valid IDE is available.
`--init`	Run Setup hooks with the `init` matcher before the session (print mode only).
`--init-only`	Run Setup and `SessionStart` hooks, then exit without starting a conversation.
`--include-hook-events`	Include hook lifecycle events in the output stream. `SessionStart` and `Setup` hook events are always included and don't need this flag.
`--include-partial-messages`	Include partial streaming events in output.
`--input-format`	Specify input format for print mode (options: `text`, `stream-json`).
`--json-schema`	Get validated JSON output matching a JSON Schema after the agent completes its workflow (print mode only).
`--maintenance`	Run Setup hooks with the `maintenance` matcher before the session (print mode only).
`--max-budget-usd`	Stop the run once estimated spend on API calls reaches this amount (print mode only).
`--max-turns`	Limit the number of agentic turns (print mode only).
`--mcp-config`	Load MCP servers from JSON files or strings (space-separated).
`--model`	Sets the model for the current session with a model alias such as `sonnet`, `opus`, `haiku`, or `fable`, or a model's full name.
`--name`, `-n`	Set a display name for the session, shown in `/resume` and the terminal title.
`--no-chrome`	Disable Chrome browser integration for this session.
`--no-session-persistence`	Disable session persistence so sessions are not saved to disk and cannot be resumed.
`--output-format`	Specify output format for print mode (options: `text`, `json`, `stream-json`).
`--permission-mode`	Begin in a specified permission mode.

`--permission-prompt-tool`	Specify an MCP tool to handle permission prompts in non-interactive mode.
`--permission-prompts`	Set who answers permission prompts in print mode.
`--plugin-dir`	Load a plugin from a directory or `.zip` archive, or several from a folder of plugins, for this session only.
`--plugin-url`	Fetch a plugin `.zip` archive from a URL for this session only.
`--print`, `-p`	Print response without interactive mode (see Agent SDK documentation for programmatic usage details).
`--prompt-suggestions`	Emit a `prompt_suggestion` message with a predicted next user prompt after each turn that generates one; very short conversations can produce none.
`--ref <branch>`	With `--environment`, base the new session's checkout on a named ref instead of local `HEAD`.
`--remote`	Deprecated alias for `--cloud`, including the existing-session form.
`--remote-control`, `--rc`	Start an interactive session with Remote Control enabled so you can also control it from claude.ai or the Claude app.
`--remote-control-session-name-prefix <prefix>`	Prefix for auto-generated Remote Control session names when no explicit name is set.
`--replay-user-messages`	Re-emit user messages from stdin back on stdout for acknowledgment.
`--restricted`	Start in restricted mode.
`--resume`, `-r`	Resume a specific session by ID or name, or show an interactive picker to choose a session.
`--safe-mode`	Start with all customizations disabled to troubleshoot a broken configuration: CLAUDE.md, skills, plugins, hooks, MCP servers, custom commands and agents, output...
`--session-id`	Use a specific session ID for the conversation (must be a valid UUID).
`--setting-sources`	Comma-separated list of setting sources to load (`user`, `project`, `local`).
`--settings`	Path to a settings JSON file or an inline JSON string.
`--strict-mcp-config`	Only use MCP servers from `--mcp-config`, ignoring all other MCP configurations.
`--system-prompt`	Replace the entire system prompt with custom text.
`--system-prompt-file`	Load system prompt from a file, replacing the default prompt.
`--system-prompt-snapshot`	Pass `off` to rebuild the system prompt on every request instead of reusing the prompt recorded on the conversation's first request, for example while you iterate on...
`--teleport`	Resume a cloud session in your local terminal.
`--teammate-mode`	Set how agent team teammates display: `in-process` (default), `auto`, `tmux`, or `iterm2`.
`--tmux`	Create a tmux session for the worktree.
`--tools`	Restrict which built-in tools Claude can use.
`--verbose`	Enable verbose logging, shows full turn-by-turn output.
`--version`, `-v`	Output the version number.
`--worktree`, `-w`	Start Claude in an isolated git worktree at `<repo>/.claude/worktrees/<name>`.
`--system-prompt`	Replaces the entire default prompt.
`--system-prompt-file`	Replaces with file contents.
`--append-system-prompt`	Appends to the default prompt.

`--append-system-prompt-file`	Appends file contents to the default prompt.
`--system-prompt-snapshot`	With `off`, rebuilds the prompt on every request.

14.4 Every keyboard shortcut

KEYS	WHAT IT DOES
`Ctrl+C`	Interrupt, or clear input.
`Ctrl+X Ctrl+K`	Stop all running background subagents in this session, and turn off artifact auto-replies for the rest of it.
`Ctrl+D`	Exit Claude Code session.
`Ctrl+G` or `Ctrl+X Ctrl+E`	Open in default text editor.
`Ctrl+L`	Redraw the screen.
`Ctrl+O`	Toggle transcript viewer.
`Ctrl+R`	Reverse search command history.
`Ctrl+V` or `Cmd+V` (iTerm2) or `Alt+V` (Windows and WSL)	Paste image from clipboard.
`Ctrl+B`	Background running tasks.
`Ctrl+T`	Toggle Claude's task checklist.
`Ctrl+S`	Stash or restore prompt.
`Ctrl+Z`	Suspend Claude Code.
`Left/Right arrows`	Cycle through dialog tabs.
`Tab`	Accept an autocomplete suggestion, or add a comment to a permission answer.
`Up/Down arrows` or `Ctrl+P` / `Ctrl+N`	Move cursor or navigate command history.
`Esc`	Interrupt Claude, or close a dialog.
`Esc` + `Esc`	Clear input draft, or rewind.
`Ctrl+Enter` or `Ctrl+X Ctrl+S`	Send queued messages now.
`Shift+Tab`, or `Alt+M` on Windows when the Node or Bun runtime doesn't enable VT input mode	Cycle permission modes.
`Option+P` (macOS) or `Alt+P` (Windows/Linux)	Switch model.
`Option+T` (macOS) or `Alt+T` (Windows/Linux)	Toggle extended thinking.
`Option+O` (macOS) or `Alt+O` (Windows/Linux)	Toggle fast mode.
`Ctrl+A`	Move cursor to start of current line.
`Ctrl+E`	Move cursor to end of current line.
`Ctrl+K`	Delete to end of line.
`Ctrl+U`	Delete from cursor to line start.
`Ctrl+W`	Delete back to previous whitespace.
`Ctrl+Y`	Paste deleted text.
`Alt+Y` (after `Ctrl+Y`)	Cycle paste history.

`Alt+B`	Move cursor back one word.
`Alt+F`	Move cursor forward one word.
`Alt+D`	Delete to end of word.
`Ctrl+_` or `Ctrl+Shift+-`	Undo last input edit.
`Ctrl+T`	Toggle syntax highlighting for code blocks.
`/` at start	Command or skill.
`!` at start	Shell mode.
`@`	File path mention.
`:`	Emoji shortcode.
`?` on empty input	Toggle the shortcut help panel.
`?`	Toggle the keyboard shortcut help panel.
`{` / `}`	Jump to the previous or next user prompt, like vim paragraph motion.
`Ctrl+E`	Toggle show all content.
`[`	Write the full conversation to your terminal's native scrollback so `Cmd+F`, tmux copy mode, and other native tools…
`v`	Write the conversation to a temporary file and open it in `$VISUAL` or `$EDITOR`.
`q`, `Ctrl+C`, `Esc`	Exit transcript view.
`Esc` or `Ctrl+[`	Enter NORMAL mode.
`i`	Insert before cursor.
`I`	Insert at beginning of line.
`a`	Insert after cursor.
`A`	Insert at end of line.
`o`	Open line below.
`O`	Open line above.
`v`	Start character-wise visual selection.
`V`	Start line-wise visual selection.
`h` / `j` / `k` / `l`	Move left/down/up/right.
`Space`	Move right.
`w`	Next word.
`e`	End of word.
`b`	Previous word.
`0`	Beginning of line.
`$`	End of line.
`^`	First non-blank character.
`gg`	Beginning of input.

`G`	Beginning of last line.
`f{char}`	Jump to next occurrence of character on the current line.
`F{char}`	Jump to previous occurrence of character on the current line.
`t{char}`	Jump to just before next occurrence of character on the current line.
`T{char}`	Jump to just after previous occurrence of character on the current line.
`;`	Repeat last f/F/t/T motion.
`,`	Repeat last f/F/t/T motion in reverse.
`%`	Jump to the matching `()`, `[]`, or `{}` bracket, starting from the next bracket on the current line when the cursor...
`/`	Open reverse history search, same as `Ctrl+R`.
`x`	Delete character.
`r{char}`	Replace character under cursor with `{char}`.
`dd`	Delete line.
`D`	Delete to end of line.
`dw` / `de` / `db`	Delete word/to end/back.
`df{char}` / `dt{char}`	Delete to and including, or up to, the next occurrence of a character on the current line.
`dj` / `dk`	Delete the current line and the line below or above.
`dgg` / `dG`	Delete from the current line to the first or last line.
`d0` / `c0` / `y0`	Delete, change, or yank from the cursor back to the beginning of the line.
`d%` / `c%` / `y%`	Delete, change, or yank from the cursor through the bracket that `%` jumps to.
`cc`	Change line.
`C`	Change to end of line.
`cw` / `ce` / `cb`	Change word/to end/back.
`s`	Substitute character: delete the character under the cursor and enter INSERT mode.
`S`	Substitute line: clear the line and enter INSERT mode.
`yy` / `Y`	Yank (copy) line.
`yw` / `ye` / `yb`	Yank word/to end/back.
`p`	Paste after cursor.
`P`	Paste before cursor.
`>>`	Indent line.
`<<`	Dedent line.
`J`	Join lines.
`u`	Undo.
`.`	Repeat last change.
`iw` / `aw`	Inner/around word.
`iW` / `aW`	Inner/around WORD (whitespace-delimited).

`i"` / `a"`	Inner/around double quotes.
`i'` / `a'`	Inner/around single quotes.
`i(` / `a(`	Inner/around parentheses.
`i[` / `a[`	Inner/around brackets.
`i{` / `a{`	Inner/around braces.
`d` / `x`	Delete selection.
`y`	Yank selection.
`c` / `s`	Change selection.
`p`	Replace selection with register contents.
`r{char}`	Replace every selected character with `{char}`.
`~` / `u` / `U`	Toggle, lowercase, or uppercase selection.
`>` / `<`	Indent or dedent selected lines.
`J`	Join selected lines.
`o`	Swap cursor and anchor.
`iw` / `aw` / `i"` /…	Select a text object.
`v` / `V`	Toggle between character-wise and line-wise, or exit.
`Space`, `Enter`, `Escape`	Dismiss the answer and return to the prompt.
`Up` / `Down`	Scroll the answer.
`Shift+Left` / `Shift+Right`	Step between this answer and your earlier `/btw` answers. `Shift+Left` moves to older answers and `Shift+Right`…
`c`	Copy the answer to your clipboard as raw Markdown.
`f`	Start a forked subagent that inherits the parent conversation plus this question and answer, so it can continue with…
`x`	Clear the list of earlier `/btw` exchanges shown above the current answer.

CHAPTER 15

Real-World Workflows, Step by Step

Knowing the commands is half the job. This chapter strings them into complete, repeatable workflows you can follow with the book open next to your keyboard.

IN THIS CHAPTER

- Fix a bug from an error message
- Add a feature safely with plan mode
- Understand a project you've never seen
- Write and run tests
- Ship a pull request
- Run long tasks in the background
- Automate with `claude -p`

15.1 Workflow 1: Fix a bug from an error message

1. **Reproduce it.**
   Run the program and copy the full error, including the lines above it.
2. **Start Claude in the project folder.**
   Type `claude`.

3. **Paste the error with context.**

   PROMPT

   When I run `npm start` I get the error below. Find the cause, explain it in plain English, then propose the smallest fix. Don't change anything else.
   [paste error]

4. **Read the explanation before approving.**
   If it doesn't make sense to you, ask "why?" until it does.

5. **Approve the edit, then ask Claude to rerun the command.**
   A fix isn't done until the error is gone.

6. **Commit.**
   Ask: "Commit this with a clear message."

15.2 Workflow 2: Add a feature with plan mode

1. **Enter plan mode.**
   Press Shift + Tab until the footer says *plan*, or type `/plan add a dark-mode toggle`.

2. **Let Claude research.**
   It reads files but can't edit them.

3. **Review the plan.**
   Push back on anything too big. Press Ctrl + G to edit the plan in your text editor.

4. **Approve.**
   Claude switches out of plan mode and starts working.

5. **Check in at milestones.**
   Ask for a short status after each step.

6. **Review with `/diff`, then `/code-review`.**

7. **Compact or clear.**
   If the session is long and you're continuing, use `/compact`. If you're done, commit and `/clear`.

15.3 Workflow 3: Learn an unfamiliar project

PROMPT

Give me a beginner-friendly tour of this project: what it does, how to run it, the main folders, and the five files I should read first. Don't edit anything.

Follow up with "trace what happens when a user clicks *Sign up*." This **trace** question is the fastest way to learn real code. When you're done, run `/init` so the next session starts with what you learned.

15.4 Workflow 4: Tests first

1. **Ask for tests that describe the behaviour you want,** and tell Claude they should fail for now.
2. **Run them and confirm they fail.**
   This proves the tests actually check something.
3. **Ask Claude to make the tests pass without changing the tests.**
4. **Run the full test suite** so you know nothing else broke.

> TIP
> Saying "don't modify the tests" stops a common shortcut, where an AI "fixes" a failing test by weakening it.

15.5 Workflow 5: Ship a pull request

1. **Branch first.**
   "Create a branch called `feature/contact-form`."
2. **Do the work** (Workflow 2).
3. **Review.**
   `/diff` → `/code-review` → `/security-review`.
4. **Commit and push.**
   "Commit in logical chunks and push the branch."
5. **Open the PR.**
   "Open a pull request with a summary and testing notes." This uses the `gh` tool (Vol. 2).

6. **Address feedback.**
   Paste reviewer comments back to Claude, or try `/autofix-pr` if your plan supports cloud sessions.

15.6 Workflow 6: Long tasks in the background

Press Ctrl + B to send a long-running command, like a dev server or test run, to the background. Use `/tasks` to check on it. To detach the whole session and get your terminal back, use `/background`. Then watch it from your shell with `claude agents`. `/subtask` hands a side job to a helper that sees the full conversation.

15.7 Workflow 7: Scripts with `claude -p`

`-p` (print mode) runs one request with no interactive screen. That makes it ideal for scripts and scheduled jobs.

TERMINAL

```
claude -p "Summarize what changed in git today in five bullets" > today.md
git diff | claude -p "Write a commit message for this diff"
```

> ⚠ **WARNING**
> In print mode nobody is around to approve prompts. Give it only the permissions it needs, with `--allowedTools` or allow rules, and never broad shell access on a machine you care about.

15.8 Daily routine checklist

- [ ] `git status` is clean before you start
- [ ] Start with `claude -c` to continue, or `claude` for a fresh task
- [ ] One task per conversation
- [ ] Use plan mode for anything touching more than two files
- [ ] `/context` when things feel slow, then `/compact` or `/clear`
- [ ] `/diff` before every commit
- [ ] Commit small and often
- [ ] End by asking for a summary of what changed and what's next

CHAPTER 16

Troubleshooting and FAQ

When something goes wrong, work through this chapter from top to bottom. Most problems are one of a dozen common causes.

IN THIS CHAPTER

- A first-aid routine for any problem
- Install and sign-in fixes
- When Claude gets slow, forgetful, or stuck
- Permission and tool problems
- Frequently asked questions

16.1 First-aid routine

1. **Read the whole message.**
   The fix is often in the last line.
2. **Run `claude doctor` in your terminal,** or `/doctor` inside a session.
3. **Update.**
   Run `claude update`, then restart.
4. **Check sign-in.**
   Run `claude auth status`.
5. **Turn on logging.**
   Restart with `claude --debug`, or run `/debug describe the problem`.
6. **Search the official docs** at code.claude.com/docs.
7. **Report it.**
   If it's a real bug, use `/bug`. It shows you what will be sent and asks you to confirm first.

16.2 Symptom table

SYMPTOM	LIKELY CAUSE	FIX
`command not found: claude`	The install folder isn't on your PATH	Open a new terminal. Run `claude doctor`. Reinstall with the native installer
Browser sign-in never finishes	Pop-up blocked or wrong browser profile	Copy the sign-in URL into the right browser. Try `claude auth login`
"Usage limit reached"	Your plan's limit for this period is used up	`/usage` to see when it resets. `/rate-limit-options` lists choices. Switch to a lighter `/model`
Answers get vague or repetitive	The context window is full of old material	`/context`, then `/compact` or `/clear`
Claude keeps redoing a mistake	The bad attempt is still in context	`/rewind` to before the mistake. Re-prompt with the missing detail
Too many permission prompts	No allow rules yet	`/permissions` to add rules, or `/fewer-permission-prompts`
Claude edits files you didn't want touched	The request was too vague	Use plan mode. Add "only change X" to the prompt. Add deny rules
MCP server shows "failed"	Bad config or expired sign-in	`/mcp` to reconnect or sign in. Check the command and path
A skill doesn't appear in `/`	Wrong folder, bad YAML, or a name clash	Check `.claude/skills/<name>/SKILL.md`. Run `/reload-skills`
Hooks don't fire	Wrong event name or matcher	Run `/hooks` to inspect them, then `--debug`
Garbled screen	The terminal display got out of sync	Ctrl + L to redraw
Pasting an image does nothing	Your terminal uses a different key	Try Ctrl + V, Cmd + V (iTerm2), or Alt + V (Windows)
Shift + Enter sends instead of adding a new line	Your terminal doesn't support it	Run `/terminal-setup`, or use \ + Enter or Ctrl + J

16.3 Frequently asked questions

Does `/clear` delete my files? No. It only clears the conversation. Your files and git history stay as they are.

Is `/compact` the same as `/clear`? No. `/compact` keeps a summary and stays on the same task. `/clear` starts from nothing.

Can Claude see my whole computer? By default it works in the folder you started it in, plus any folders you add with `/add-dir`. It asks before it does anything outside its allowed permissions.

Does it remember me between sessions? Only through `CLAUDE.md`, auto memory, and the conversations you resume. A fresh session doesn't automatically know what you discussed yesterday.

What does it cost? It depends on your plan and changes over time. See the Costs chapter, and run `/usage` for your own numbers. This manual doesn't quote prices, because they change.

Is my code used for training? That depends on your account type and settings. Check Anthropic's current privacy settings for your plan. Don't take a third-party guide's word for it.

Can I use it on my phone? You can use Claude Code on the web and in the Claude app, and `/remote-control` lets you drive a session running on your computer. See the "Beyond the Terminal" chapter.

CHAPTER 17

Hooks, Settings, and Environment Reference

This is the deep reference for people who want to automate Claude Code. You don't need it on day one. The tables were generated from Anthropic's hooks reference (code.claude.com/docs/en/hooks) and environment-variable reference (code.claude.com/docs/en/env-vars) in October 2026.

IN THIS CHAPTER

- Every hook event and when it fires
- A worked hook example
- Where settings files live and which one wins
- Every documented environment variable

17.1 Settings files and precedence

FILE	SCOPE	COMMIT TO GIT?
Managed settings (set by IT)	Whole organisation, can't be overridden	Not applicable
`.claude/settings.local.json`	You, in this project	No, keep it private
`.claude/settings.json`	Everyone on this project	Yes
`~/.claude/settings.json`	You, in every project	Not applicable

When two files disagree, the more specific one wins: managed, then command-line flags, then local project, then shared project, then user. **Deny** rules beat allow rules wherever they come from.

17.2 Hook events

A **hook** is a command that runs automatically when an event fires. Use `/hooks` to see the hooks you have, or `/update-config` to ask Claude to add one.

EVENT	FIRES...
`SessionStart`	When a session begins or resumes.
`Setup`	When you start Claude Code with `--init-only`, or with `--init` or `--maintenance` in `-p` mode.
`UserPromptSubmit`	When a prompt is submitted, before Claude processes it.
`UserPromptExpansion`	When a user-typed command expands into a prompt, before it reaches Claude.
`PreToolUse`	Before a tool call executes.
`PermissionRequest`	When a tool call needs a permission decision.
`PermissionDenied`	When auto mode denies a tool call, including denials without a classifier verdict.
`PostToolUse`	After a tool call succeeds.
`PostToolUseFailure`	After a tool call fails.
`PostToolBatch`	After a full batch of parallel tool calls resolves, before the next model call.
`Notification`	When Claude Code sends a notification.
`MessageDisplay`	While assistant message text is displayed.
`SubagentStart`	When a subagent is spawned.
`SubagentStop`	When a subagent finishes.
`TaskCreated`	When a task is being created via `TaskCreate`.
`TaskCompleted`	When a task is being marked as completed.
`Stop`	When Claude finishes responding.
`StopFailure`	When the turn ends due to an API error.
`TeammateIdle`	When an agent team teammate is about to go idle.
`InstructionsLoaded`	When a CLAUDE.md or `.claude/rules/*.md` file is loaded into context.
`ConfigChange`	When a configuration file changes during a session.
`CwdChanged`	When the working directory changes, for example when Claude executes a `cd` command.
`DirectoryAdded`	When a working directory is added mid-session via `/add-dir` or the SDK `register_repo_root` control request.
`FileChanged`	When a watched file changes on disk.
`WorktreeCreate`	When a worktree is being created via `--worktree`, `isolation: "worktree"`, or for a background session.
`WorktreeRemove`	When a worktree that a `WorktreeCreate` hook created is being removed.
`PreCompact`	Before context compaction.
`PostCompact`	After context compaction completes.
`PreModelSwitch`	Before Claude Code applies a model switch that you or a client requested.

`PostModelSwitch`	After the session's model changes, including changes Claude Code makes on its own, such as restoring the model when you resume a session.
`Elicitation`	When an MCP server requests user input during a tool call.
`ElicitationResult`	After a user responds to an MCP elicitation, before the response is sent back to the server.
`SessionEnd`	When a session terminates.

17.3 Worked example: format every file Claude edits

.CLAUDE/SETTINGS.JSON

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write"
}
        ]
      }
    ]
  }
}
```

> ⚠ WARNING
>
> Hooks run with **your** user permissions and no prompt. Only add hooks you've read and understand, and be careful with hooks that come inside repositories you cloned.

The hook receives details of the tool call as JSON on its input. `jq` pulls out the edited file's path, so Prettier formats only that file. You need `jq` and Prettier installed.

17.4 Environment variables

Set these in your shell or in the `env` block of `settings.json`. Most people never need more than a handful. Search this table when a guide or error message mentions one.

VARIABLE	WHAT IT DOES
`ANTHROPIC_API_KEY`	API key sent as `X-Api-Key` header.
`ANTHROPIC_AUTH_TOKEN`	Custom value for the `Authorization` header (the value you set here will be prefixed with `Bearer `).
`ANTHROPIC_AWS_API_KEY`	Workspace API key for Claude Platform on AWS, generated in the AWS Console.
`ANTHROPIC_AWS_BASE_URL`	Override the Claude Platform on AWS endpoint URL. Use for custom regions or when routing through an LLM...

`ANTHROPIC_AWS_WORKSPACE_ID`	Required for Claude Platform on AWS. Sent on every request as the `anthropic-workspace-id` header.
`ANTHROPIC_BASE_URL`	Override the API endpoint to route requests through a proxy or gateway.
`ANTHROPIC_BEDROCK_BASE_URL`	Override the Amazon Bedrock endpoint URL. Use for custom Amazon Bedrock endpoints or when routing through...
`ANTHROPIC_BEDROCK_MANTLE_BASE_URL`	Override the Amazon Bedrock Mantle endpoint URL. See Mantle endpoint.
`ANTHROPIC_BEDROCK_REGION_PREFIX`	Cross-region inference profile prefix (`us` , `eu` , `apac` , `jp` , `au` , or `global`) Claude Code tries...
`ANTHROPIC_BEDROCK_SERVICE_TIER`	Amazon Bedrock service tier (`default` , `flex` , or `priority`).
`ANTHROPIC_BETAS`	Comma-separated list of additional `anthropic-beta` header values to include in API requests.
`ANTHROPIC_CUSTOM_HEADERS`	Custom headers to add to requests (`Name: Value` format, newline-separated for multiple headers).
`ANTHROPIC_CUSTOM_MODEL_OPTION`	Model ID to add as a custom entry in the `/model` picker.
`ANTHROPIC_CUSTOM_MODEL_OPTION_DESCRIPTION`	Display description for the custom model entry in the `/model` picker.
`ANTHROPIC_CUSTOM_MODEL_OPTION_NAME`	Display name for the custom model entry in the `/model` picker.
`ANTHROPIC_CUSTOM_MODEL_OPTION_SUPPORTED_CAPABILITIES`	Comma-separated list of capabilities the custom model supports, for example `effort,thinking` .
`ANTHROPIC_DEFAULT_FABLE_MODEL`	Model ID that the `fable` alias resolves to, and the ID Claude Code recognizes as a Fable model for...
`ANTHROPIC_DEFAULT_FABLE_MODEL_DESCRIPTION`	Display description for the pinned Fable model in the `/model` picker.
`ANTHROPIC_DEFAULT_FABLE_MODEL_NAME`	Display name for the pinned Fable model in the `/model` picker.
`ANTHROPIC_DEFAULT_FABLE_MODEL_SUPPORTED_CAPABILITIES`	Comma-separated list of capabilities the pinned Fable model supports, for example `effort,thinking` .
`ANTHROPIC_DEFAULT_HAIKU_MODEL`	Model ID that the `haiku` alias resolves to, also used for background functionality.
`ANTHROPIC_DEFAULT_HAIKU_MODEL_DESCRIPTION`	Display description for the pinned Haiku model in the `/model` picker.
`ANTHROPIC_DEFAULT_HAIKU_MODEL_NAME`	Display name for the pinned Haiku model in the `/model` picker.
`ANTHROPIC_DEFAULT_HAIKU_MODEL_SUPPORTED_CAPABILITIES`	Comma-separated list of capabilities the pinned Haiku model supports, for example `effort,thinking` .
`ANTHROPIC_DEFAULT_MODEL`	Model that new sessions start on by default.
`ANTHROPIC_DEFAULT_OPUS_MODEL`	Model ID that the `opus` alias resolves to, and that `opusplan` uses while Plan Mode is active.
`ANTHROPIC_DEFAULT_OPUS_MODEL_DESCRIPTION`	Display description for the pinned Opus model in the `/model` picker.

`ANTHROPIC_DEFAULT_OPUS_MODEL_NAME`	Display name for the pinned Opus model in the `/model` picker.
`ANTHROPIC_DEFAULT_OPUS_MODEL_SUPPORTED_CAPABILITIES`	Comma-separated list of capabilities the pinned Opus model supports, for example `effort,thinking`.
`ANTHROPIC_DEFAULT_SONNET_MODEL`	Model ID that the `sonnet` alias resolves to, and that `opusplan` uses when Plan Mode is not active.
`ANTHROPIC_DEFAULT_SONNET_MODEL_DESCRIPTION`	Display description for the pinned Sonnet model in the `/model` picker.
`ANTHROPIC_DEFAULT_SONNET_MODEL_NAME`	Display name for the pinned Sonnet model in the `/model` picker.
`ANTHROPIC_DEFAULT_SONNET_MODEL_SUPPORTED_CAPABILITIES`	Comma-separated list of capabilities the pinned Sonnet model supports, for example `effort,thinking`.
`ANTHROPIC_FEDERATION_RULE_ID`	Federation rule ID for Workload Identity Federation.
`ANTHROPIC_FOUNDRY_API_KEY`	API key for Microsoft Foundry authentication (see Microsoft Foundry).
`ANTHROPIC_FOUNDRY_AUTH_TOKEN`	Bearer token for Microsoft Foundry authentication, such as a Microsoft Entra access token.
`ANTHROPIC_FOUNDRY_BASE_URL`	Full base URL for the Microsoft Foundry resource (for example,...
`ANTHROPIC_FOUNDRY_RESOURCE`	Microsoft Foundry resource name (for example, `my-resource`).
`ANTHROPIC_MODEL`	Name of the model setting to use (see Model Configuration).
`ANTHROPIC_ORGANIZATION_ID`	Organization ID for Workload Identity Federation.
`ANTHROPIC_PROFILE`	Name of the Anthropic profile to authenticate with, such as one created by `ant auth login` or by signing...
`ANTHROPIC_SMALL_FAST_MODEL`	[DEPRECATED] Name of Haiku-class model for background tasks.
`ANTHROPIC_SMALL_FAST_MODEL_AWS_REGION`	Override AWS region for the Haiku-class model when using Amazon Bedrock or Amazon Bedrock Mantle.
`ANTHROPIC_VERTEX_BASE_URL`	Override Google Cloud's Agent Platform endpoint URL. Use for custom Google Cloud's Agent Platform...
`ANTHROPIC_VERTEX_PROJECT_ID`	GCP project ID that Google Cloud's Agent Platform requests are addressed to.
`ANTHROPIC_WORKSPACE_ID`	Workspace ID for workload identity federation.
`API_FORCE_IDLE_TIMEOUT`	Override the 5-minute body idle timeout that aborts a streaming model response when no bytes arrive.
`API_TIMEOUT_MS`	Timeout for API requests in milliseconds (default: 600000, or 10 minutes; maximum: 2147483647).
`AWS_BEARER_TOKEN_BEDROCK`	Amazon Bedrock API key for authentication (see Amazon Bedrock API keys).
`BASH_DEFAULT_TIMEOUT_MS`	Default timeout for a foreground Bash or PowerShell tool command, in milliseconds (default: 120000, or 2...
`BASH_MAX_OUTPUT_LENGTH`	Maximum number of characters of bash output that Claude Code reads back into a command's result (default:...

`BASH_MAX_TIMEOUT_MS`	Maximum timeout the model can set for a foreground Bash or PowerShell tool command, in milliseconds...
`BETA_TRACING_ENDPOINT`	OTLP/HTTP endpoint for detailed beta tracing: with `ENABLE_BETA_TRACING_DETAILED=1`, logs and traces go...
`CCR_FORCE_BUNDLE`	Set to `1` to force `claude --cloud` to bundle and upload your local repository instead of cloning from...
`CLAUDECODE`	Set to `1` in subprocesses Claude Code spawns (Bash and PowerShell tools, tmux sessions, hook commands,...
`CLAUDE_AFK_COUNTDOWN_MS`	How many milliseconds before auto-continue the on-screen countdown appears on an unanswered...
`CLAUDE_AFK_TIMEOUT_MS`	How many milliseconds of idle time before an unanswered `AskUserQuestion` dialog auto-continues without you.
`CLAUDE_AGENT_SDK_DISABLE_BUILTIN_AGENTS`	Set to `1` to disable all built-in subagent types such as Explore and Plan.
`CLAUDE_AGENT_SDK_MCP_NO_PREFIX`	Set to `1` to skip the `mcp__<server>__` prefix on tool names from SDK-created MCP servers.
`CLAUDE_ASYNC_AGENT_STALL_TIMEOUT_MS`	Stall timeout in milliseconds for subagents.
`CLAUDE_AUTOCOMPACT_PCT_OVERRIDE`	Set the percentage (1-100) of the auto-compact window at which auto-compaction triggers.
`CLAUDE_AUTO_BACKGROUND_TASKS`	Set to `1` to force-enable automatic backgrounding of long-running agent tasks.
`CLAUDE_AX_PREPARK_MS`	In screen reader mode, how many milliseconds Claude Code waits before it writes a new or changed line.
`CLAUDE_AX_SCREEN_READER`	Set to `1` to render screen-reader friendly output: flat text without decorative borders or animations.
`CLAUDE_AX_STARTUP_QUIET_MS`	In screen reader mode, how many milliseconds Claude Code holds the first interface render after the...
`CLAUDE_BASH_MAINTAIN_PROJECT_WORKING_DIR`	Return to the original working directory after each Bash or PowerShell command in the main session.
`CLAUDE_BYTE_STREAM_IDLE_TIMEOUT_MS`	Timeout in milliseconds for the byte-level streaming idle watchdog; when set, it takes precedence over...
`CLAUDE_CLIENT_PRESENCE_FILE`	Path to a file that an external tool, such as a screen-lock listener, creates when you unlock your screen...
`CLAUDE_CODE_ACCESSIBILITY`	Set to `1` to keep the native terminal cursor visible and disable the inverted-text cursor indicator.
`CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD`	Set to `1` to load memory files from directories specified with `--add-dir`.
`CLAUDE_CODE_ALT_SCREEN_FULL_REPAINT`	Set to `1` to repaint the entire screen on every frame in fullscreen rendering instead of sending...
`CLAUDE_CODE_ALWAYS_ENABLE_EFFORT`	Set to `1` to send the effort parameter with every request, even when Claude Code does not recognize the...
`CLAUDE_CODE_API_KEY_HELPER_TTL_MS`	Interval in milliseconds at which credentials should be refreshed (when using `apiKeyHelper`).
`CLAUDE_CODE_ARTIFACT_AUTO_OPEN`	Set to `0` to stop Claude Code from opening the browser automatically when a new artifact is published.

`CLAUDE_CODE_ARTIFACT_COMMENTS`	Set to `0` to stop Claude reading and replying to comments on an artifact.
`CLAUDE_CODE_ARTIFACT_COMMENTS_AUTOREACT`	Set to `0` to stop Claude replying on its own to comments sent to it.
`CLAUDE_CODE_ARTIFACT_PUBLISH_EDITS`	Set to `0` to stop Claude from applying small changes to an artifact's file as part of publishing it.
`CLAUDE_CODE_ATTRIBUTION_HEADER`	Set to `0` to omit the attribution block, which carries the client version and a prompt fingerprint, from…
`CLAUDE_CODE_AUTO_BACKGROUND_WORKER_CHECKIN_SECONDS`	Removed in v2.1.283. Use `CLAUDE_CODE_WORKER_CHECKIN_SCHEDULE` instead.
`CLAUDE_CODE_AUTO_COMPACT_WINDOW`	Set the auto-compact window in tokens, from `100000` to `1000000`.
`CLAUDE_CODE_AUTO_CONNECT_IDE`	Override automatic IDE connection.
`CLAUDE_CODE_AUTO_MODE_SERVER`	Controls whether Claude Code asks the server to review auto mode actions.
`CLAUDE_CODE_AWS_CHAIN_RESOLVE_TIMEOUT_MS`	Time in milliseconds Claude Code waits for the AWS default credential provider chain to produce…
`CLAUDE_CODE_BASH_EDIT_DIFF`	Set to `0` to turn off the diff of the files that changed while a Bash command ran, or `1` to record it in…
`CLAUDE_CODE_BG_TASKS_REPORT_RUNNING`	Set to `0` to make a non-interactive session report an idle status to its host at every turn end, even…
`CLAUDE_CODE_BRIDGE_SESSION_ID`	Set automatically in Bash tool and hook command subprocesses while the session has an active Remote…
`CLAUDE_CODE_BS_AS_CTRL_BACKSPACE`	Set to `0` to make Claude Code read the `0x08` byte, also written `^H`, as plain Backspace, or `1` to read…
`CLAUDE_CODE_CERT_STORE`	Comma-separated list of CA certificate sources for TLS connections. `bundled` is the Mozilla CA set…
`CLAUDE_CODE_CHILD_SESSION`	Set to `1` in subprocesses Claude Code spawns via the Bash, PowerShell, and Monitor tools, hook commands,…
`CLAUDE_CODE_CLIENT_CERT`	Path to client certificate file for mTLS authentication.
`CLAUDE_CODE_CLIENT_KEY`	Path to client private key file for mTLS authentication.
`CLAUDE_CODE_CLIENT_KEY_PASSPHRASE`	Passphrase for encrypted CLAUDE_CODE_CLIENT_KEY (optional).
`CLAUDE_CODE_CONNECT_TIMEOUT_MS`	Removed in v2.1.186 and now a no-op.
`CLAUDE_CODE_DEBUG_LOGS_DIR`	Override the debug log file path.
`CLAUDE_CODE_DEBUG_LOG_LEVEL`	Minimum log level written to the debug log file.
`CLAUDE_CODE_DISABLE_1M_CONTEXT`	Set to `1` to turn off 1M context window support.
`CLAUDE_CODE_DISABLE_ADAPTIVE_THINKING`	Set to `1` to disable adaptive reasoning on Opus 4.6 and Sonnet 4.6 and fall back to the fixed thinking…
`CLAUDE_CODE_DISABLE_ADMIN_ENV_UNION`	Set to `1` to stop Claude Code from merging managed settings `env` blocks per key across admin sources, so…
`CLAUDE_CODE_DISABLE_ADVISOR_TOOL`	Set to `1` to disable the advisor tool.

`CLAUDE_CODE_DISABLE_AGENT_VIEW`	Set to `1` to turn off background agents and agent view: `claude agents`, `--bg`, `/background`, and the…
`CLAUDE_CODE_DISABLE_ALTERNATE_SCREEN`	Set to `1` to disable fullscreen rendering and use the classic main-screen renderer.
`CLAUDE_CODE_DISABLE_ARTIFACT`	Set to `1` to turn off the Artifact tool, which publishes session output as a private web page on claude.ai.
`CLAUDE_CODE_DISABLE_ATTACHMENTS`	Set to `1` to disable attachment processing.
`CLAUDE_CODE_DISABLE_AUTH_REFRESH_LOCK`	Set to `1` to make a Claude Code process run its `gcpAuthRefresh` or `awsAuthRefresh` command itself…
`CLAUDE_CODE_DISABLE_AUTO_MEMORY`	Set to `1` to disable auto memory.
`CLAUDE_CODE_DISABLE_BACKGROUND_TASKS`	Set to `1` to disable all background task functionality, including the `run_in_background` parameter on…
`CLAUDE_CODE_DISABLE_BEDROCK_CONTENT_TYPE_DEFAULT`	Set to `1` to stop Claude Code from treating an Amazon Bedrock streaming response with a missing or empty…
`CLAUDE_CODE_DISABLE_BEDROCK_CONTENT_TYPE_GUARD`	Set to `1` to skip the check that an Amazon Bedrock streaming response carries the…
`CLAUDE_CODE_DISABLE_BG_EXIT_HANDOFF`	Set to `1` to stop a background session's running background shell commands, dynamic workflows, and, as of…
`CLAUDE_CODE_DISABLE_BG_SHELL_PRESSURE_REAP`	Set to `1` to stop Claude Code from terminating background shell commands under memory pressure.
`CLAUDE_CODE_DISABLE_BUNDLED_SKILLS`	Set to `1` to disable the skills and workflows included with Claude Code: bundled skills and workflows are…
`CLAUDE_CODE_DISABLE_CFC_PROMPT`	Set to `1` to keep the Claude in Chrome browser tools available while omitting the Chrome section of the…
`CLAUDE_CODE_DISABLE_CLAUDE_MDS`	Set to `1` to prevent loading any CLAUDE.md memory files into context, including user, project, and auto…
`CLAUDE_CODE_DISABLE_CRON`	Set to `1` to disable scheduled tasks.
`CLAUDE_CODE_DISABLE_DANGEROUS_RM_TIMEOUT`	Set to `1` to turn off the time limit on critical-path removal prompts.
`CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS`	Set to `1` to strip pre-release `anthropic-beta` request headers, the body fields that pair with them, and…
`CLAUDE_CODE_DISABLE_EXPLORE_PLAN_AGENTS`	Set to `1` to disable the built-in Explore and Plan subagents.
`CLAUDE_CODE_DISABLE_FAST_MODE`	Set to `1` to disable fast mode.
`CLAUDE_CODE_DISABLE_FEEDBACK_SURVEY`	Set to `1` to disable the "How is Claude doing?" session quality surveys.
`CLAUDE_CODE_DISABLE_FILE_CHECKPOINTING`	Set to `1` to disable file checkpointing.
`CLAUDE_CODE_DISABLE_FILE_MENTIONS`	Set to `1` to stop Claude Code from expanding `@` mentions of files and directories.
`CLAUDE_CODE_DISABLE_GIT_INSTRUCTIONS`	Set to `1` to remove built-in commit and PR workflow instructions and the git status snapshot from…
`CLAUDE_CODE_DISABLE_INLINE_SHELL_RM_PROMPT`	Set to `1` to stop Claude Code from reading scripts passed to a shell with `-c`, such as `bash -c 'rm -rf…
`CLAUDE_CODE_DISABLE_LEGACY_MODEL_REMAP`	Set to `1` to prevent automatic remapping of Opus 4 and Opus 4.1 to the current Opus version on the…

`CLAUDE_CODE_DISABLE_MODEL_ACCESS_FALLBACK`	Set to `1` to stop Claude Code on Amazon Bedrock and Google Cloud's Agent Platform from switching to an...
`CLAUDE_CODE_DISABLE_MOUSE`	Set to `1` to disable mouse tracking in fullscreen rendering.
`CLAUDE_CODE_DISABLE_MOUSE_CLICKS`	Set to `1` to disable click, drag, and hover handling in fullscreen rendering while keeping mouse-wheel...
`CLAUDE_CODE_DISABLE_MTLS_RELOAD_ON_STALE_CONNECTION`	Set to `1` to stop Claude Code from re-reading the mTLS client certificate and key when an API request...
`CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC`	Set to any non-empty value, such as `1`, to disable nonessential network traffic: auto-updates, telemetry,...
`CLAUDE_CODE_DISABLE_NONSTREAMING_FALLBACK`	Set to `1` to disable the non-streaming fallback when a streaming request fails mid-stream.
`CLAUDE_CODE_DISABLE_NOTIFICATION_PRESENCE_CHECK`	Set to `1` to send the `PushNotification` tool's desktop notification even while you are typing in or...
`CLAUDE_CODE_DISABLE_OFFICIAL_MARKETPLACE_AUTOINSTALL`	Set to `1` to disable automatic registration of the official plugin marketplace.
`CLAUDE_CODE_DISABLE_PERMISSION_PROMPT_NOTIFY_HOOKS`	Set to `1` to stop Claude Code from running your `Notification` hooks for unanswered permission requests...
`CLAUDE_CODE_DISABLE_POLICY_SKILLS`	Set to `1` to skip loading skills from the system-wide managed skills directory.
`CLAUDE_CODE_DISABLE_POWERSHELL_CMD_RM_DENY`	Set to `1` to turn off the PowerShell tool check that denies the `cmd` built-ins `rd`, `rmdir`, `del`, and...
`CLAUDE_CODE_DISABLE_REFUSAL_FALLBACK`	Set to `1` to turn off the automatic model switch when a safety classifier flags a request, the behavior...
`CLAUDE_CODE_DISABLE_STRUCTURED_OUTPUTS`	Set to `1` to stop Claude Code from sending the structured-output `output_config.format` field and the...
`CLAUDE_CODE_DISABLE_SUBSTITUTION_RM_PROMPT`	Set to `1` to turn off the critical-path check for a recursive `rm` whose target is entirely the output of...
`CLAUDE_CODE_DISABLE_TERMINAL_TITLE`	Set to `1` to disable automatic terminal title updates based on conversation context.
`CLAUDE_CODE_DISABLE_THINKING`	Set to `1` to omit the `thinking` parameter from API requests entirely.
`CLAUDE_CODE_DISABLE_UNKNOWN_MODEL_WINDOW_ENFORCEMENT`	Set to `1` to skip proactive auto-compaction when Claude Code doesn't recognize the model ID, such as an...
`CLAUDE_CODE_DISABLE_VIRTUAL_SCROLL`	Set to `1` to disable virtual scrolling in fullscreen rendering and render every message in the transcript.
`CLAUDE_CODE_DISABLE_WEB_FETCH`	Set to `1` to turn off the WebFetch tool.
`CLAUDE_CODE_DISABLE_WINDOWS_SHELL_LAUNCHER`	Set to `1` to start PowerShell tool commands on Windows directly instead of through the `cmd.exe` launcher.
`CLAUDE_CODE_DISABLE_WORKFLOWS`	Set to `1` to disable workflows.
`CLAUDE_CODE_EFFORT_LEVEL`	Set the effort level for supported models.
`CLAUDE_CODE_EMIT_SESSION_STATE_EVENTS`	Set to `1` to add `session_state_changed` messages, which carry the session's state, to the message stream.
`CLAUDE_CODE_ENABLE_AUTO_MODE`	Accepted for compatibility with older releases and has no effect.

`CLAUDE_CODE_ENABLE_AWAY_SUMMARY`	Override session recap availability.
`CLAUDE_CODE_ENABLE_BACKGROUND_PLUGIN_REFRESH`	Set to `1` to refresh plugin state at turn boundaries in non-interactive mode after a background install…
`CLAUDE_CODE_ENABLE_CFC`	Set to `1` to start a CLI session with Chrome integration on, or `0` to start with it off.
`CLAUDE_CODE_ENABLE_FEEDBACK_SURVEY_FOR_OTEL`	Set to `1` to route the “How is Claude doing?” session quality survey to your own OpenTelemetry collector…
`CLAUDE_CODE_ENABLE_FINE_GRAINED_TOOL_STREAMING`	Controls whether tool call inputs stream from the API as Claude generates them.
`CLAUDE_CODE_ENABLE_GATEWAY_MODEL_DISCOVERY`	Set to `1` to populate the `/model` picker from your gateway’s `/v1/models` endpoint when…
`CLAUDE_CODE_ENABLE_OPUS_4_7_FAST_MODE`	Removed in v2.1.142, when the fast mode default moved from Opus 4.6 to Opus 4.7.
`CLAUDE_CODE_ENABLE_PROMPT_SUGGESTION`	Set to `false` to turn off prompt suggestions, the grayed-out predictions that appear in your prompt input.
`CLAUDE_CODE_ENABLE_TASKS`	Selects which task-tracking tools Claude Code provides in sessions that have them.
`CLAUDE_CODE_ENABLE_TELEMETRY`	Set to `1` to enable OpenTelemetry data collection for metrics and logging.
`CLAUDE_CODE_ENABLE_TODO_TOOLS`	Set to `1` to get the task-tracking tools on every model.
`CLAUDE_CODE_EXIT_AFTER_STOP_DELAY`	Time in milliseconds to wait after the query loop becomes idle before automatically exiting.
`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS`	Set to `1` to enable agent teams.
`CLAUDE_CODE_EXTRA_BODY`	JSON object to merge into the top level of every API request body.
`CLAUDE_CODE_FILE_READ_MAX_OUTPUT_TOKENS`	Override the default token limit for file reads, which is 25,000 tokens.
`CLAUDE_CODE_FORCE_SESSION_PERSISTENCE`	Set to `1` to force transcript persistence, prompt history, and `claude agents` registration even when…
`CLAUDE_CODE_FORCE_STRIKETHROUGH`	Set to `1` to force strikethrough rendering for `~~text~~` in Claude’s responses when your terminal…
`CLAUDE_CODE_FORCE_SYNC_OUTPUT`	Set to `1` to force-enable DEC private mode 2026 synchronized output when your terminal supports it but is…
`CLAUDE_CODE_FORCE_TERMINAL_IMAGES`	Set to `1` to draw mod `Image` elements as pictures when your terminal draws kitty graphics protocol…
`CLAUDE_CODE_FORK_SUBAGENT`	Controls fork mode, which lets Claude spawn forked subagents itself and is on by default in interactive…
`CLAUDE_CODE_FORWARD_SUBAGENT_TEXT`	Set to `1` to emit subagent text and thinking blocks in `claude -p --output-format stream-json` output,…
`CLAUDE_CODE_GATEWAY_HINT_HEADERS`	Set to `1` to send the gateway hint headers, such as `x-claude-code-request-class` and…
`CLAUDE_CODE_GATEWAY_MODEL_DISCOVERY_TIMEOUT_MS`	Timeout in milliseconds for the gateway model discovery request that…
`CLAUDE_CODE_GIT_BASH_PATH`	Windows only: path to the Git Bash executable (`bash.exe`).

`CLAUDE_CODE_GLOB_HIDDEN`	Set to `false` to exclude dotfiles from results when Claude invokes the Glob tool.
`CLAUDE_CODE_GLOB_NO_IGNORE`	Set to `false` to make the Glob tool respect `.gitignore` patterns.
`CLAUDE_CODE_GLOB_TIMEOUT_SECONDS`	Timeout in seconds for Glob tool file discovery.
`CLAUDE_CODE_GOAL_CHECKIN_MINUTES`	How many minutes background work can keep an active goal waiting before Claude Code asks Claude to check...
`CLAUDE_CODE_GZIP_REQUEST_BODIES`	Set to `0` to turn off gzip compression of Claude API, telemetry, and artifact publish request bodies sent...
`CLAUDE_CODE_HIDE_CWD`	Set to `1` to hide the working directory in the startup logo.
`CLAUDE_CODE_IDE_HOST_OVERRIDE`	Override the host address used to connect to the IDE extension.
`CLAUDE_CODE_IDE_SKIP_AUTO_INSTALL`	Set to `1` to skip auto-installation of IDE extensions.
`CLAUDE_CODE_IDE_SKIP_VALID_CHECK`	Set to `1` to skip validation of IDE lockfile entries during connection.
`CLAUDE_CODE_IDLE_COMPACT_MIN_TOKENS`	Context size, in tokens, at which idle compaction applies to a conversation (default: `200000`).
`CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS`	How many subagents can be running in one session before the Agent tool refuses to spawn another (default: 20).
`CLAUDE_CODE_MAX_CONTEXT_TOKENS`	Override the context window size Claude Code assumes for the active model.
`CLAUDE_CODE_MAX_MCP_DESCRIPTION_LENGTH`	Maximum length in characters of MCP tool descriptions and MCP server instructions that Claude Code sends...
`CLAUDE_CODE_MAX_OUTPUT_TOKENS`	Set the maximum number of output tokens for most requests.
`CLAUDE_CODE_MAX_RETRIES`	Override the number of times to retry failed API requests (default: 10).
`CLAUDE_CODE_MAX_SUBAGENTS_PER_SESSION`	Removed in v2.1.224 and now a no-op.
`CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH`	Number of subagent layers allowed below the main conversation (default: 3).
`CLAUDE_CODE_MAX_TOOL_USE_CONCURRENCY`	Maximum number of read-only tools and subagents that can execute in parallel (default: 10).
`CLAUDE_CODE_MAX_TURNS`	Cap the number of agentic turns when no explicit limit is passed.
`CLAUDE_CODE_MAX_WEB_SEARCHES_PER_SESSION`	Cap on WebSearch calls (default: 200).
`CLAUDE_CODE_MCP_ALLOWLIST_ENV`	Set to `1` to spawn stdio MCP servers with only a safe baseline environment plus the server's configured...
`CLAUDE_CODE_MCP_AUTO_BACKGROUND_MS`	Elapsed time in milliseconds before a still-running MCP tool call moves to a background task (default:...
`CLAUDE_CODE_MCP_STARTUP_WAIT_MS`	How long in milliseconds the first turn of a non-interactive session waits for MCP servers that are still...
`CLAUDE_CODE_MCP_TOOL_IDLE_TIMEOUT`	Idle timeout in milliseconds for MCP tool calls.
`CLAUDE_CODE_MESSAGING_SOCKET`	Set by Claude Code, not by you: in sessions that bind an inbox socket, Claude Code exports that socket's...
`CLAUDE_CODE_MESSAGING_TOKEN`	Set by Claude Code, not by you: in sessions that bind an inbox socket, Claude Code exports this...
`CLAUDE_CODE_NATIVE_CURSOR`	Set to `1` to show the terminal's own cursor at the input caret instead of a drawn block.

`CLAUDE_CODE_NEW_INIT`	Set to `1` to make `/init` run an interactive setup flow.
`CLAUDE_CODE_NONBLOCKING_STDOUT`	Set to `1` to write terminal output through a second non-blocking file descriptor, so a terminal that…
`CLAUDE_CODE_NONSTREAMING_TIMEOUT_RETRIES`	Limit how many times Claude Code re-sends a non-streaming request that times out.
`CLAUDE_CODE_NO_FLICKER`	Set to `1` to enable fullscreen rendering, a research preview that reduces flicker and keeps memory flat…
`CLAUDE_CODE_OAUTH_REFRESH_TOKEN`	OAuth refresh token for Claude.ai authentication.
`CLAUDE_CODE_OAUTH_SCOPES`	Space-separated OAuth scopes the refresh token was issued with, such as `"user:profile user:inference…
`CLAUDE_CODE_OAUTH_TOKEN`	OAuth access token for claude.ai authentication.
`CLAUDE_CODE_OPUS_4_6_FAST_MODE_OVERRIDE`	Removed in v2.1.160 and now a no-op.
`CLAUDE_CODE_OTEL_CONTENT_MAX_LENGTH`	Maximum length of content-bearing OpenTelemetry attributes (model responses, tool content, system prompts,…
`CLAUDE_CODE_OTEL_DIAG_STDERR`	Set to `1` to write OpenTelemetry exporter diagnostic errors to stderr.
`CLAUDE_CODE_OTEL_FLUSH_TIMEOUT_MS`	Timeout in milliseconds for flushing pending OpenTelemetry spans (default: 5000).
`CLAUDE_CODE_OTEL_HEADERS_HELPER_DEBOUNCE_MS`	Interval for refreshing dynamic OpenTelemetry headers in milliseconds (default: 1740000 / 29 minutes).
`CLAUDE_CODE_OTEL_SHUTDOWN_TIMEOUT_MS`	Timeout in milliseconds for the OpenTelemetry exporter to finish on shutdown (default: 2000).
`CLAUDE_CODE_OVERLOADED_RETRY_BASE_DELAY_MS`	Starting delay in milliseconds, in place of the default 500, of the exponential backoff between automatic…
`CLAUDE_CODE_OVERLOADED_RETRY_MAX_DELAY_MS`	Ceiling in milliseconds, in place of the default 32000, on the exponential backoff between automatic…
`CLAUDE_CODE_PACKAGE_MANAGER_AUTO_UPDATE`	Set to `1` to let Claude Code run your package manager's upgrade command in the background when a new…
`CLAUDE_CODE_PERFORCE_MODE`	Set to `1` to enable Perforce-aware write protection.
`CLAUDE_CODE_PLUGIN_CACHE_DIR`	Override the plugins root directory.
`CLAUDE_CODE_PLUGIN_DIRS`	Plugin directories to load for the session, each loaded the way a `--plugin-dir` flag loads it.
`CLAUDE_CODE_PLUGIN_DIR_WATCH`	Controls whether Claude Code reloads a mod when the mod's files change.
`CLAUDE_CODE_PLUGIN_GIT_TIMEOUT_MS`	Timeout in milliseconds for cloning or refreshing a plugin marketplace (default: 120000).
`CLAUDE_CODE_PLUGIN_KEEP_MARKETPLACE_ON_FAILURE`	Set to `1` to skip the re-clone attempt and keep using the existing marketplace checkout when a…
`CLAUDE_CODE_PLUGIN_PREFER_HTTPS`	Set to `1` to clone GitHub `owner/repo` shorthand sources over HTTPS instead of SSH. Applies to plugin…
`CLAUDE_CODE_PLUGIN_SEED_DIR`	Path to one or more read-only plugin seed directories, separated by `:` on Unix or `;` on Windows.
`CLAUDE_CODE_POWERSHELL_RESPECT_EXECUTION_POLICY`	Set to `1` to stop Claude Code from passing `-ExecutionPolicy Bypass` when spawning PowerShell for tool…

`CLAUDE_CODE_PRINT_BG_WAIT_CEILING_MS`	Ceiling in milliseconds on idle waiting for background work, such as subagents and workflows, after the…
`CLAUDE_CODE_PROCESS_WRAPPER`	Launch the processes Claude Code starts from its own binary, such as the background service that hosts…
`CLAUDE_CODE_PROJECT_DIR_NAME`	Set together with `CLAUDE_CONFIG_DIR` to choose the `projects/` directory name Claude Code stores that…
`CLAUDE_CODE_PROMPT_CACHE_TTL`	Set `5m` or `1h`, the only values Claude Code accepts, to choose the prompt cache TTL for the main…
`CLAUDE_CODE_PROPAGATE_TRACEPARENT`	Set to `1` to propagate W3C trace context when `ANTHROPIC_BASE_URL` points at a custom proxy.
`CLAUDE_CODE_PROVIDER_MANAGED_BY_HOST`	Set by host platforms that embed Claude Code and manage model provider routing on its behalf.
`CLAUDE_CODE_PROXY_RESOLVES_HOSTS`	Set to `1` to allow the proxy to perform DNS resolution instead of the caller.
`CLAUDE_CODE_REMOTE`	Set automatically to `true` when Claude Code is running as a cloud session.
`CLAUDE_CODE_REMOTE_SESSION_ID`	Set automatically in cloud sessions to the current session's ID. Read this to construct a link back to the…
`CLAUDE_CODE_RESTRICTED`	Set to `1` to start the session in restricted mode, the same as passing `--restricted`.
`CLAUDE_CODE_RESUME_INTERRUPTED_TURN`	Set to `1` to automatically resume if the previous session ended mid-turn.
`CLAUDE_CODE_RESUME_INTERRUPTED_TURN_MAX_AGE_MS`	Maximum age in milliseconds of the last transcript message for a session that ended mid-turn to continue…
`CLAUDE_CODE_RESUME_PROMPT`	Override the continuation message Claude Code sends to Claude when `CLAUDE_CODE_RESUME_INTERRUPTED_TURN`…
`CLAUDE_CODE_RETRY_WATCHDOG`	Set to `1` for unattended sessions such as eval harnesses, CI jobs, or remote workers.
`CLAUDE_CODE_RETRY_WATCHDOG_MAX_WAIT_MS`	Maximum time in milliseconds that each API request spends waiting out `429` and `529` errors when…
`CLAUDE_CODE_SAFE_MODE`	Set to `1` to start in safe mode: CLAUDE.md, skills, plugins, hooks, MCP servers, custom commands and…
`CLAUDE_CODE_SCRIPT_CAPS`	JSON object limiting how many times specific scripts may be invoked per session when…
`CLAUDE_CODE_SCROLL_SPEED`	Set the mouse wheel scroll multiplier in fullscreen rendering.
`CLAUDE_CODE_SEND_FEEDBACK`	Set to `0` to turn off Claude-drafted feedback for a session.
`CLAUDE_CODE_SESSIONEND_HOOKS_TIMEOUT_MS`	Override the time budget in milliseconds for SessionEnd hooks.
`CLAUDE_CODE_SESSION_ID`	Set automatically to the current session ID in Bash and PowerShell tool subprocesses, hook command…
`CLAUDE_CODE_SHELL`	Set the shell Claude Code uses to run Bash tool commands.
`CLAUDE_CODE_SHELL_PREFIX`	Command prefix that wraps shell commands Claude Code spawns: Bash tool calls, hook commands, status line…
`CLAUDE_CODE_SIMPLE`	Set to `1` to run with a minimal system prompt and only the Bash, file read, and file edit tools.

`CLAUDE_CODE_SIMPLE_SYSTEM_PROMPT`	Choose between Claude Code's full system prompt and a shorter one with abbreviated tool descriptions.
`CLAUDE_CODE_SKIP_ANTHROPIC_AWS_AUTH`	Skip client-side authentication for Claude Platform on AWS, for gateways that sign requests themselves.
`CLAUDE_CODE_SKIP_AWS_CRED_CACHE`	Set to `1` to turn off the in-process cache of credentials resolved from the AWS default credential...
`CLAUDE_CODE_SKIP_BEDROCK_AUTH`	Skip AWS authentication for Amazon Bedrock (for example, when using an LLM gateway).
`CLAUDE_CODE_SKIP_FAST_MODE_NETWORK_ERRORS`	Set to `1` to treat a failed fast mode availability check as available, for networks that block the...
`CLAUDE_CODE_SKIP_FAST_MODE_ORG_CHECK`	Set to `1` to skip the client-side fast mode availability check, for proxies that intercept the check's...
`CLAUDE_CODE_SKIP_FOUNDRY_AUTH`	Skip Azure authentication for Microsoft Foundry, for a proxy or gateway that injects its own...
`CLAUDE_CODE_SKIP_MANTLE_AUTH`	Skip AWS authentication for Amazon Bedrock Mantle (for example, when using an LLM gateway).
`CLAUDE_CODE_SKIP_MODEL_ACCESS_MEMORY`	The startup model checks on Amazon Bedrock and Google Cloud's Agent Platform remember on this machine...
`CLAUDE_CODE_SKIP_PROMPT_HISTORY`	Set to `1` to skip writing prompt history and session transcripts to disk.
`CLAUDE_CODE_SKIP_VERTEX_AUTH`	Skip Google authentication for Google Cloud's Agent Platform (for example, when using an LLM gateway).
`CLAUDE_CODE_STARTUP_FAILURE_RESULTS`	Set to `1` to have a session started with `--output-format stream-json` write a result message naming why...
`CLAUDE_CODE_STOP_HOOK_BLOCK_CAP`	Maximum number of consecutive times a Stop or SubagentStop hook may block the turn from ending before...
`CLAUDE_CODE_SUBAGENT_MODEL`	The default model for subagents, agent team teammates, and workflow agents that aren't assigned a model...
`CLAUDE_CODE_SUBAGENT_MODEL_FORCE`	Set to `1` to force one model onto subagents, teammates, and workflow agents.
`CLAUDE_CODE_SUBAGENT_PROMPT_CACHE_TTL`	Set `5m` or `1h`, the only values Claude Code accepts, to choose the prompt cache TTL for requests outside...
`CLAUDE_CODE_SUBPROCESS_ENV_SCRUB`	Set to `1` to strip credentials from the environments of the subprocesses Claude Code starts, such as Bash...
`CLAUDE_CODE_SYNC_PLUGIN_INSTALL`	Set to `1` in non-interactive mode (the `-p` flag) to wait for plugin installation to complete before the...
`CLAUDE_CODE_SYNC_PLUGIN_INSTALL_TIMEOUT_MS`	Timeout in milliseconds for synchronous plugin installation.
`CLAUDE_CODE_SYNC_SKILLS`	Set to `1` in non-interactive mode with the `-p` flag to make Claude Code download the skills enabled for...
`CLAUDE_CODE_SYNC_SKILLS_INSTALL_TIMEOUT_MS`	Timeout in milliseconds for the skills resync that runs mid-session when an app built on the Agent SDK...
`CLAUDE_CODE_SYNC_SKILLS_WAIT_TIMEOUT_MS`	Timeout in milliseconds for the first query to wait for the initial skill list when...
`CLAUDE_CODE_SYNTAX_HIGHLIGHT`	Set to `false` to disable syntax highlighting in diff output.
`CLAUDE_CODE_TASK_LIST_ID`	Share a task list across sessions.

`CLAUDE_CODE_TEAM_TEARDOWN_PARK_TIMEOUT_MS`	Override, in milliseconds, how long a non-interactive session waits at exit for its agent team to finish…
`CLAUDE_CODE_TMPDIR`	Override the temp directory used for internal temp files.
`CLAUDE_CODE_TMUX_TRUECOLOR`	Set to any non-empty value, such as `1`, to allow 24-bit truecolor output inside tmux. **Setting it to `0`…
`CLAUDE_CODE_TOOL_MEMORY_CGROUP_EXCLUDE`	On Linux and WSL, set to a comma-separated list of the kinds of processes Claude Code excludes from the…
`CLAUDE_CODE_TOOL_MEMORY_LIMIT`	On Linux and WSL, set to a size such as `4G` to cap the memory that Bash and PowerShell tool commands can…
`CLAUDE_CODE_TRANSCRIPT_LOCAL_GC`	Set to `1` to limit how large the transcript file of a long `-p` or Agent SDK session grows.
`CLAUDE_CODE_USER_DIALOG_TIMEOUT_MS`	Deadline in milliseconds before Claude Code cancels a dialog it forwards to a remote client such as a…
`CLAUDE_CODE_USE_ANTHROPIC_AWS`	Use Claude Platform on AWS.
`CLAUDE_CODE_USE_BEDROCK`	Use Amazon Bedrock.
`CLAUDE_CODE_USE_FOUNDRY`	Use Microsoft Foundry.
`CLAUDE_CODE_USE_MANTLE`	Use the Amazon Bedrock Mantle endpoint.
`CLAUDE_CODE_USE_NATIVE_FILE_SEARCH`	Set to `1` to discover custom commands, subagents, and output styles using Node.js file APIs instead of…
`CLAUDE_CODE_USE_POWERSHELL_TOOL`	Controls the PowerShell tool.
`CLAUDE_CODE_USE_VERTEX`	Use Google Cloud's Agent Platform.
`CLAUDE_CODE_WEBFETCH_CACHE_TTL_MS`	Set to the number of milliseconds WebFetch keeps each fetched URL's response cached.
`CLAUDE_CODE_WEBFETCH_DEADLINE_MS`	Upper bound in milliseconds on how long WebFetch waits for a page to download, including any redirects it…
`CLAUDE_CODE_WEB_SEARCH_REFILLS_PER_HOUR`	Rate at which a session's WebSearch limit refills, in calls per hour.
`CLAUDE_CODE_WORKER_CHECKIN_SCHEDULE`	When `CLAUDE_AUTO_BACKGROUND_TASKS` is set to `1`, how long Claude Code waits before each reminder to…
`CLAUDE_CODE_WORKFLOW_MAX_CONCURRENT_AGENTS`	How many agents a single workflow run executes at once, from `1` to `256`.
`CLAUDE_CODE_WORKFLOW_PREFIX_STAGGER_MS`	Upper bound in milliseconds on how long a workflow agent waits for a same-prefix sibling's first response…
`CLAUDE_CODE_YOU_SHOULD_KNOW_DEBUG`	Set to `1` to show, above the prompt, a line for each of the last three checks the You should know mod…
`CLAUDE_CONFIG_DIR`	Override the configuration directory (default: `~/.claude`).
`CLAUDE_DISABLE_ADOPT`	Set to `1` to stop in-flight background work instead of carrying it over when you background a session by…
`CLAUDE_EFFORT`	Set automatically in Bash tool subprocesses and hook commands to the effort level in effect when the…
`CLAUDE_ENABLE_BYTE_WATCHDOG`	Set to `1` to force-enable the byte-level streaming idle watchdog, or set to `0` to force-disable it. `0`…
`CLAUDE_ENABLE_BYTE_WATCHDOG_BEDROCK`	Set to `1` to enable the byte-level streaming idle watchdog on Amazon Bedrock `vnd.amazon.eventstream`…

`CLAUDE_ENABLE_STREAM_WATCHDOG`	Set to `0` to force-disable the event-level streaming idle watchdog, or set to `1` to force-enable it.
`CLAUDE_ENV_FILE`	Path to a shell script whose contents Claude Code runs before each Bash command in the same shell process,...
`CLAUDE_JOB_DIR`	Set by Claude Code in each background session to that session's `~/.claude/jobs/<id>` directory.
`CLAUDE_PID`	Claude Code sets this to its own process ID in the subprocesses it spawns: Bash and PowerShell tool...
`CLAUDE_REMOTE_CONTROL_SESSION_NAME_PREFIX`	Prefix for auto-generated Remote Control session names when no explicit name is provided.
`CLAUDE_STREAM_FIRST_BYTE_TIMEOUT_MS`	Deadline in milliseconds for the first response byte of a streaming request, on the connections where the...
`CLAUDE_STREAM_IDLE_TIMEOUT_MS`	Timeout in milliseconds before the event- and byte-level streaming idle watchdogs close a stalled connection.
`CLAUDE_SUBAGENT_BG_SHELL_MAX_MS`	Removed in v2.1.260 and now a no-op.
`DEBUG`	Set to `1` to enable debug mode, equivalent to launching with `--debug`.
`DISABLE_AUTOUPDATER`	Set to `1` to disable automatic background updates.
`DISABLE_AUTO_COMPACT`	Set to `1` to disable automatic compaction when approaching the context limit, along with idle compaction.
`DISABLE_COMPACT`	Set to `1` to disable all compaction: both automatic compaction and the manual `/compact` command.
`DISABLE_COST_WARNINGS`	Set to `1` to disable cost warning messages.
`DISABLE_DOCTOR_COMMAND`	Set to `1` to hide the `/doctor` setup checkup skill and its `/checkup` alias.
`DISABLE_ERROR_REPORTING`	Set to any non-empty value, such as `1`, to opt out of error reporting. **Setting it to `0` or `false`...
`DISABLE_EXTRA_USAGE_COMMAND`	Set to `1` to hide the `/usage-credits` command that lets users purchase additional usage beyond rate limits.
`DISABLE_FEEDBACK_COMMAND`	Set to `1` to disable the `/feedback` command and Claude-drafted feedback.
`DISABLE_GROWTHBOOK`	Set to `1` or `true` to disable GrowthBook feature-flag fetching and use code defaults for every flag.
`DISABLE_INSTALLATION_CHECKS`	Set to `1` to disable installation warnings.
`DISABLE_INSTALL_GITHUB_APP_COMMAND`	Set to `1` to hide the `/install-github-app` command.
`DISABLE_INTERLEAVED_THINKING`	Set to `1` to prevent sending the interleaved-thinking beta header.
`DISABLE_LOGIN_COMMAND`	Set to `1` to hide the `/login` command.
`DISABLE_LOGOUT_COMMAND`	Set to `1` to hide the `/logout` command.
`DISABLE_PROMPT_CACHING`	Set to `1` to disable prompt caching for all models (takes precedence over per-model settings).
`DISABLE_PROMPT_CACHING_FABLE`	Set to `1` to disable prompt caching for Fable models.
`DISABLE_PROMPT_CACHING_HAIKU`	Set to `1` to disable prompt caching for the default Haiku model, wherever it runs.

`DISABLE_PROMPT_CACHING_OPUS`	Set to `1` to disable prompt caching for the default Opus model.
`DISABLE_PROMPT_CACHING_SONNET`	Set to `1` to disable prompt caching for the default Sonnet model.
`DISABLE_TELEMETRY`	Set to any non-empty value, such as `1`, to opt out of telemetry. **Setting it to `0` or `false` still...
`DISABLE_UPDATES`	Set to `1` to block all updates including manual `claude update` and `claude install`.
`DISABLE_UPGRADE_COMMAND`	Set to `1` to hide the `/upgrade` command.
`DO_NOT_TRACK`	Set to `1` to opt out of telemetry, with the same effect as `DISABLE_TELEMETRY`, including on feature-flag...
`ENABLE_BETA_TRACING_DETAILED`	Set to `1`, and set `BETA_TRACING_ENDPOINT` to your OTLP/HTTP collector endpoint, to turn on detailed beta...
`ENABLE_CLAUDEAI_MCP_SERVERS`	Set to `false` to stop Claude Code from fetching claude.ai MCP servers.
`ENABLE_PROMPT_CACHING_1H`	Set to `1` to request a 1-hour prompt cache TTL instead of the default 5 minutes.
`ENABLE_PROMPT_CACHING_1H_BEDROCK`	Deprecated.
`ENABLE_TOOL_SEARCH`	Controls MCP tool search.
`FALLBACK_FOR_ALL_PRIMARY_MODELS`	Set to any non-empty value, such as `1`, to make Claude Code stop retrying on repeated overload errors for...
`FORCE_AUTOUPDATE_PLUGINS`	Set to `1` to force plugin auto-updates even when the main auto-updater is disabled via `DISABLE_AUTOUPDATER`.
`FORCE_HYPERLINK`	Set to `1` to enable clickable OSC 8 hyperlinks when your terminal supports them but isn't auto-detected,...
`FORCE_PROMPT_CACHING_5M`	Set to `1` to force the 5-minute prompt cache TTL even when 1-hour TTL would otherwise apply.
`HTTP_PROXY`	Specify HTTP proxy server for network connections.
`HTTPS_PROXY`	Specify HTTPS proxy server for network connections.
`IS_DEMO`	Set to any non-empty value, such as `1`, to enable demo mode: hides your email and organization name from...
`MAX_MCP_OUTPUT_TOKENS`	Maximum number of tokens allowed in MCP tool responses (default: 25000).
`MAX_STRUCTURED_OUTPUT_RETRIES`	Number of attempts Claude Code allows when the model's response fails validation against the...
`MAX_THINKING_TOKENS`	Fixed token budget for extended thinking.
`MCP_CLIENT_SECRET`	OAuth client secret for MCP servers that require pre-configured credentials.
`MCP_CONNECTION_NONBLOCKING`	Controls whether startup waits for MCP servers to connect before the first query.
`MCP_CONNECT_TIMEOUT_MS`	How long blocking MCP startup waits, in milliseconds, for the connection batch before snapshotting the...
`MCP_DISCOVERY_CACHE`	Turns the MCP discovery cache on or off.
`MCP_DISCOVERY_CACHE_MAX_STALE_S`	Maximum age, in seconds, of a discovery-cache entry (default: 14400, or 4 hours).
`MCP_DISCOVERY_CACHE_STRIKES`	At a start where a discovery-cache entry is older than `MCP_DISCOVERY_CACHE_TTL_S`, Claude Code refreshes...
`MCP_DISCOVERY_CACHE_TTL_S`	Seconds for which Claude Code uses a discovery-cache entry without refreshing it (default: 900).

`MCP_OAUTH_CALLBACK_PORT`	Fixed port for the OAuth redirect callback, as an alternative to `--callback-port` when adding an MCP...
`MCP_PROTOCOL_NEGOTIATION`	On the v2 MCP client runtime only, whether Claude Code probes servers for MCP protocol revision...
`MCP_REMOTE_SERVER_CONNECTION_BATCH_SIZE`	Maximum number of remote MCP servers (HTTP/SSE) to connect in parallel during startup (default: 20).
`MCP_SDK_GENERATION`	Pin which MCP client runtime this process connects to MCP servers with: `v1`, built on MCP TypeScript SDK...
`MCP_SERVER_CONNECTION_BATCH_SIZE`	Maximum number of local MCP servers (stdio) to connect in parallel during startup (default: 3).
`MCP_TIMEOUT`	Timeout in milliseconds for MCP server startup (default: 30000, or 30 seconds).
`MCP_TOOL_TIMEOUT`	Timeout in milliseconds for MCP tool execution (default: 100000000, about 28 hours).
`NO_PROXY`	List of domains and IPs to which requests will be directly issued, bypassing proxy.
`OTEL_ATTRIBUTE_VALUE_LENGTH_LIMIT`	Standard OpenTelemetry SDK limit on attribute value length.
`OTEL_LOG_ASSISTANT_RESPONSES`	Set to `1` to include the model's response text on `assistant_response` OpenTelemetry log events.
`OTEL_LOG_MANAGED_SETTINGS`	Set to `1` to add the redacted managed settings, and a SHA-256 digest of the settings before redaction, to...
`OTEL_LOG_RAW_API_BODIES`	Emit Anthropic Messages API request and response JSON as `api_request_body` / `api_response_body` log events.
`OTEL_LOG_TOOL_CONTENT`	Set to `1` to include tool content in the `tool.output` OpenTelemetry span event.
`OTEL_LOG_TOOL_DETAILS`	Set to `1` to include tool input arguments; MCP server names; user-authored workflow names; raw error...
`OTEL_LOG_USER_PROMPTS`	Set to `1` to include user prompt text in OpenTelemetry traces and logs.
`OTEL_METRICS_INCLUDE_ACCOUNT_UUID`	Set to `false` to exclude account UUID from metrics attributes (default: included).
`OTEL_METRICS_INCLUDE_ENTRYPOINT`	Set to `true` to include the session entrypoint in metrics attributes (default: excluded).
`OTEL_METRICS_INCLUDE_REPOSITORY`	Set to `true` to tag OpenTelemetry metrics and events with `vcs.*` attributes identifying the session's...
`OTEL_METRICS_INCLUDE_RESOURCE_ATTRIBUTES`	As of v2.1.161, Claude Code attaches `OTEL_RESOURCE_ATTRIBUTES` keys to metric datapoint labels.
`OTEL_METRICS_INCLUDE_SESSION_ID`	Set to `false` to exclude session ID from metrics attributes (default: included).
`OTEL_METRICS_INCLUDE_VERSION`	Set to `true` to include Claude Code version in metrics attributes (default: excluded).
`SLASH_COMMAND_TOOL_CHAR_BUDGET`	Override the character budget for skill metadata shown to the Skill tool.
`TASK_MAX_OUTPUT_LENGTH`	Removed in v2.1.277 and now a no-op, together with the `TaskOutput` tool it sized.
`USE_BUILTIN_RIPGREP`	Set to `0` to use system-installed `rg` instead of `rg` included with Claude Code.

`VERTEX_REGION_CLAUDE_3_5_HAIKU`	Override region for Claude 3.5 Haiku when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_3_5_SONNET`	Override region for Claude 3.5 Sonnet when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_3_7_SONNET`	Override region for Claude 3.7 Sonnet when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_4_0_OPUS`	Override region for Claude Opus 4 when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_4_0_SONNET`	Override region for Claude Sonnet 4 when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_4_1_OPUS`	Override region for Claude Opus 4.1 when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_4_5_OPUS`	Override region for Claude Opus 4.5 when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_4_5_SONNET`	Override region for Claude Sonnet 4.5 when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_4_6_OPUS`	Override region for Claude Opus 4.6 when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_4_6_SONNET`	Override region for Claude Sonnet 4.6 when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_4_7_OPUS`	Override region for Claude Opus 4.7 when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_4_8_OPUS`	Override region for Claude Opus 4.8 when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_5_5_OPUS`	Override region for Claude Opus 5.5 when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_5_5_SONNET`	Override region for Claude Sonnet 5.5 when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_5_OPUS`	Override region for Claude Opus 5 when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_5_SONNET`	Override region for Claude Sonnet 5 when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_FABLE_5`	Override region for Claude Fable 5 when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_FABLE_5_1`	Override region for Claude Fable 5.1 when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_HAIKU_4_5`	Override region for Claude Haiku 4.5 when using Google Cloud's Agent Platform.
`VERTEX_REGION_CLAUDE_HAIKU_5_5`	Override region for Claude Haiku 5.5 when using Google Cloud's Agent Platform.
`ANTHROPIC_API_KEY`, `AWS_SECRET_ACCESS_KEY`	Removes it.
`NPM_TOKEN`, `DB_PASSWORD`	Removes it, because the name looks like a credential.
`DATABASE_URL` **that contains a password`**	Removes it, because the value looks like a credential.
`PIP_INDEX_URL` **or** `NPM_CONFIG_REGISTRY` **that contains a password`**	Keeps the URL and cuts the username and password from it.

`GITHUB_TOKEN`, `GH_TOKEN`, `GH_ENTERPRISE_TOKEN`, `GITHUB_ENTERPRISE_TOKEN`	Leaves it in place, so that `gh` and scripts that call the GitHub API keep working.
`HTTP_PROXY`, `HTTPS_PROXY`	Leaves it in place, including a username and password in the URL. The sandbox can set these variables…
`GIT_CONFIG_COUNT`, `GIT_CONFIG_KEY_<n>`, `GIT_CONFIG_VALUE_<n>`	Leaves it in place, whatever it holds.

Glossary

Agent	An AI that can take actions, such as reading and editing files or running commands, in a loop until a task is finished, rather than only replying with text.
Agentic loop	The cycle an agent repeats: gather context, plan, act, check the result.
Auto mode	A permission mode where a separate safety classifier reviews Claude's actions instead of asking you about each one.
Bypass permissions	A permission mode (`--dangerously-skip-permissions`) that skips all checks. Meant only for isolated, disposable environments.
Checkpoint	An automatic snapshot taken before Claude edits files, which you can return to with `/rewind`.
Classifier	A second AI model that judges whether an action in auto mode matches what you asked for.
CLAUDE.md	A Markdown file of instructions Claude reads at the start of every session.
CLI	Command-line interface: a program you control by typing commands in a terminal.
Context window	The amount of conversation and file content Claude can consider at once.
Current directory	The folder your terminal is "in." Claude Code works on the files in this folder. Also called the working directory.
Diff	A view showing exactly which lines of a file were added, removed, or changed.
Git	A tool that records the history of your files so you can review changes and go back in time.
Hook	A command you configure to run automatically at a set moment, such as after every file edit.

Markdown	A simple text format that uses symbols like `#` and `-` for headings and lists.
MCP	Model Context Protocol, an open standard for connecting AI tools to other apps and data.
MCP server	A connector that gives an AI new tools, such as access to Notion, GitHub, or a database.
Model	The specific AI "brain" doing the work, e.g. Opus (most capable), Sonnet (balanced), Haiku (fastest).
PATH	The list of folders your terminal searches when you type a program's name.
Permission mode	The setting that decides which actions Claude can take without asking you.
Plan mode	A permission mode in which Claude researches and proposes a plan but makes no edits until you approve.
Plugin	An installable bundle of skills, subagents, hooks, and MCP servers.
Prompt	The request you type to Claude. Also, the terminal's "ready for input" marker.
Prompt injection	Hidden instructions in a web page, file, or tool result that try to trick an AI into doing something you didn't ask for.
Sandbox	An isolated environment that limits which files and network addresses a command can reach.
Skill	A saved set of instructions, in a `SKILL.md` file, that Claude can use automatically or that you can run as a `/command`.
Subagent	A specialist helper with its own instructions and tools that Claude can delegate work to.
Terminal	A text window for typing commands to your computer. Also called a shell, console, or command prompt.
Token	A small chunk of text, roughly three-quarters of a word, used to measure AI usage.
Worktree	A separate working copy of a Git repository, so parallel tasks don't step on each other.